CHAPTER 5 • CI/CD FOR ML + CONSOLIDATION ~15 Min

Student

Chapters 3 and 4 got AskMyNotes to a real CI + tracked-models baseline. What does Chapter 5 add?

Claude • AI Assistant

Chapter 5 is the **capstone assembly chapter**. Everything from Chapters 1-4 comes together into one working end-to-end system, then this chapter teaches you to communicate that system fluently in interviews.

Three big additions on top of Chapter 4: (a) **data versioning with DVC** so models are truly reproducible; (b) **the three-pipeline architecture** (code CI / training / serving CD) with **contract tests** gating model promotion; (c) **the hands-on lab** with a six-milestone walkthrough, plus the **interview-ready consolidation** of the entire Week 7 curriculum. By the end of this chapter, AskMyNotes is a production-baseline ML Python project template, and you have the rehearsed 90-second walkthrough answer that proves you built it.

WHAT CHAPTER 5 COVERS

5.1 CI/CD for ML. The three-pipeline architecture (code CI per commit / training per cron-manual-drift / serving CD per @production change) and why coupling them is an anti-pattern. **DVC** for large-data versioning - tiny `.dvc` pointer files in Git, actual blobs in S3/GCS/Azure remote, `dvc push` / `dvc pull` mirroring Git's push/pull.

Wiring DVC hashes as MLflow tags for full lineage. **DVC pipelines** (`dvc.yaml` + `dvc repro`) as a lightweight reactive-execution middle ground.

Model contract tests: automated gates against `@candidate` before promotion - schema + offline quality + latency + fairness + drift + backwards-compat. The ML equivalent of unit tests blocking bad code.

5.2 ML Pipeline Automation. The DAG mental model for training pipelines. Orchestrator landscape: **GitHub Actions** (simplest, for teams already on GitHub), **Prefect** (Python-native, friendly UI), **Airflow** (mature, heavy), **Metaflow** (ML-first, Netflix-born), **Kubeflow** (K8s-native, large orgs).

Same pipeline shown in three tools with tradeoffs called out. Trigger types (cron / manual / data change / drift alert / upstream event).

A complete production-ready `.github/workflows/train.yml` with matrix builds, GPU runners, and CI-driven registration + promotion.

5.3 Full Pipeline Lab. AskMyNotes as one finished system - complete repo layout (~25 Python files + 4 workflow files + 3 config files), the 12-command fresh-clone bring-up sequence, three data + control flows overlaid (training / serving / promotion), full lineage from any production prediction back to git SHA + DVC hash + hyperparameters. Three integration points to verify: (a) code CI does NOT trigger training, (b) training triggers serving only via alias + restart, (c) contract tests actually block bad models.

Complete production-ready Dockerfile.

5.4 MLflow + CI Lab. The hands-on walkthrough with exact commands. **Six milestones (M1-M6)** with commands + expected outputs + verification: M1 one tracked run, M2 sweep of 5 runs, M3 winner registered, M4 contract tests pass, M5 alias promoted to @production, M6 serving loads by alias.

Bonus: zero-code-change swap to v2 and one-command rollback to v1. Every command has a matching CI equivalent from Section 5.2's `train.yml` .

5.5 Consolidation + Interview-Ready Summaries. The one-paragraph story of Week 7. Six chapter recap paragraphs (skimmable in 5 minutes each).

30 canonical interview questions across Git, GitHub, testing, CI/CD, MLOps with sharp framings for each. Common trap questions with the wrong-vs-right framing.

Five specific portfolio artifacts to have ready on GitHub. **The rehearsed 90-second walkthrough answer** that ties the whole system together.

5.6 Chapter 5 + Week 7 Review. 16-item Chapter 5 vocabulary. 10-item self-check. The final end-to-end exercise that closes out Week 7. Week 7 closing note.

THE ASKMYNOTES END-STATE

Student

What state does AskMyNotes end this chapter in?

Claude • AI Assistant

CONCEPT CARD

By the end of Chapter 5, AskMyNotes has:

- **DVC-tracked datasets** (`data/raw_pdfs.dvc` , `data/golden_qa_v1.jsonl.dvc`) with blobs in a configured remote
- **`dvc.yaml`** + `params.yaml` defining the reactive training pipeline

- **Full contract test suite** at `tests/contract/` covering schema + quality + latency + fairness + backwards-compat
- **`.github/workflows/train.yml`** with cron + manual dispatch + GPU runners + matrix over embedding models + integrated MLflow logging + contract-gated promotion
- **12-command fresh-clone bring-up** documented in README, taking under 15 minutes
- **Full lineage chain**: every production prediction traceable back to registered model version -> MLflow run -> git SHA -> DVC data hash -> hyperparameters
- **Zero-code-change model swap** operational (alias flip + rolling restart)
- **Rollback in one command** (alias flip back + restart)
- **Five README badges** signaling CI + coverage + quality + Docker + Python version
- **Rehearsed 90-second interview walkthrough** and answers to the 30 canonical questions

That combination is a production-baseline ML Python project. Every practice here is directly transferable to any Python ML product you'll build in your career.

REAL WORLD APPLICATION

This is the state at which teams say 'we have MLOps'. Not because they've adopted every tool - they haven't - but because the plumbing between experiments and production is trustworthy.

Adding drift monitoring (Evidently), a feature store (Feast), or full Kubeflow Pipelines is a next-step upgrade, not a prerequisite. The system you've built through Chapters 1-5 is already Google MLOps Maturity Level 1 with clear paths to Level 2.

⚠ WARNING

Chapter 5 is where the temptation to skip labs is highest - the labs feel repetitive when you've read the theory. Skip them and you'll fumble the interview walkthrough.

Do M1-M6 hands-on; do the bring-up on a fresh machine; do the swap + rollback. The muscle memory is what converts 'I've read about MLOps' into 'I've operated MLOps'.

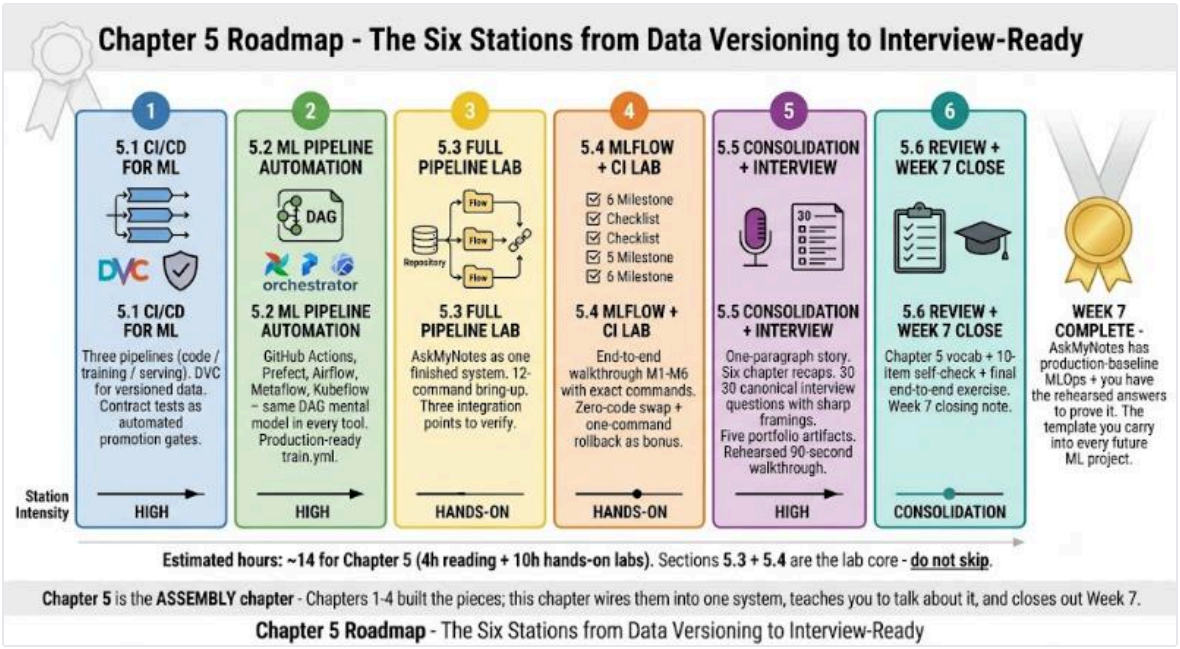
EXAM TIP CARD

Chapter 5's unique value for interviews: **the rehearsed walkthrough**. Chapters 3 and 4 give you the individual skills (pytest, GitHub Actions, MLflow); Chapter 5 gives you the connecting narrative that ties them together into one story.

Interviewers who probe MLOps roles are listening for that narrative, not just the buzzwords.

THE CHAPTER 5 ROADMAP

Six stations of the Chapter 5 journey - see the roadmap figure below. Each subsection is roughly 2 hours reading + 2 hours hands-on. Sections 5.3 and 5.4 are lab-heavy - budget for typing commands, not just reading them.



Chapter 5 Roadmap - The Six Stations from Data Versioning to Interview-Ready

Section 5 At-a-Glance - What Each Subsection Covers

Sub	Title	Focus	Emphasis
5.1	CI/CD for ML	Three-pipeline architecture, DVC data versioning, model contract tests as promotion gates	High
5.2	ML Pipeline Automation	DAG mental model, orchestrator landscape (Actions / Prefect / Airflow / Metaflow), production train.yml	High
5.3	Full Pipeline Lab	AskMyNotes as one finished system: repo tree, 12-command bring-up, three integration points, Dockerfile	Hands-on
5.4	MLflow + CI Lab	Six milestones M1-M6 with exact commands, zero-code swap + one-command rollback	Hands-on
5.5	Consolidation + Interview	Week 7 story, chapter recaps, 30 canonical interview Qs, portfolio artifacts, rehearsed 90-second walkthrough	High
5.6	Chapter 5 + Week 7 Review	16-item vocab, 10-item self-check, final end-to-end exercise, Week 7 close	Consolidation

```
bash — Section 5 preview - the CI/CD-for-ML toolbox you'll master
```

```
# ---- DVC (5.1) ----
pip install 'dvc[s3]'
dvc init
dvc remote add -d storage s3://my-bucket/askmynotes
dvc add data/raw_pdfs
git add data/raw_pdfs.dvc && git commit -m 'data: track raw_pdfs'
dvc push

# ---- DVC pipelines (5.1) ----
dvc repro                              # reactive execution: skip up-to-date
stages
dvc dag                                # print DAG
dvc metrics diff HEAD~1                # compare metrics vs previous commit

# ---- Contract tests (5.1) ----
pytest tests/contract/ -v              # all contract tests against
@candidate

# ---- Training pipeline via GitHub Actions (5.2) ----
gh workflow run train.yml -f chunk_size=800 -f promote_to=staging
gh run watch                           # tail latest run live

# ---- Full system bring-up (5.3, 12 commands) ----
# See Section 5.3 code block for the complete sequence

# ---- MLflow + CI lab (5.4) ----
python experiments/track_run.py --chunk-size 500
python experiments/promote_best_model.py --target-alias candidate
python -c "from mlflow import MlflowClient; \
  MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 1)"

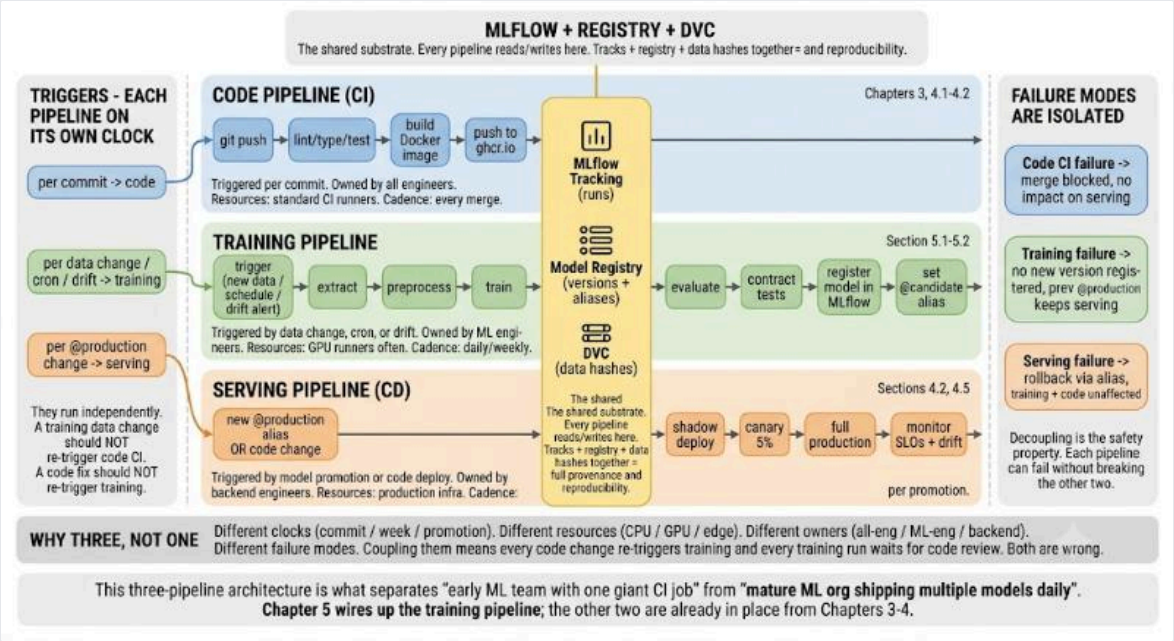
# ---- Rollback in one command (the demo everyone remembers) ----
python -c "from mlflow import MlflowClient; \
  MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 1)"
# + rolling restart of serving. Total time: 10 seconds.

# ---- Files you will create in Chapter 5 ----
#  data/*.dvc                            DVC data pointers
#  dvc.yaml + params.yaml                DVC pipeline definition + params
#  pipelines/{extract,chunk,embed,evaluate,register}.py  training stages
#  tests/contract/test_retriever_contract.py             ML unit tests
#  .github/workflows/train.yml            orchestrated training pipeline
#  Dockerfile                            multi-stage runtime image
```

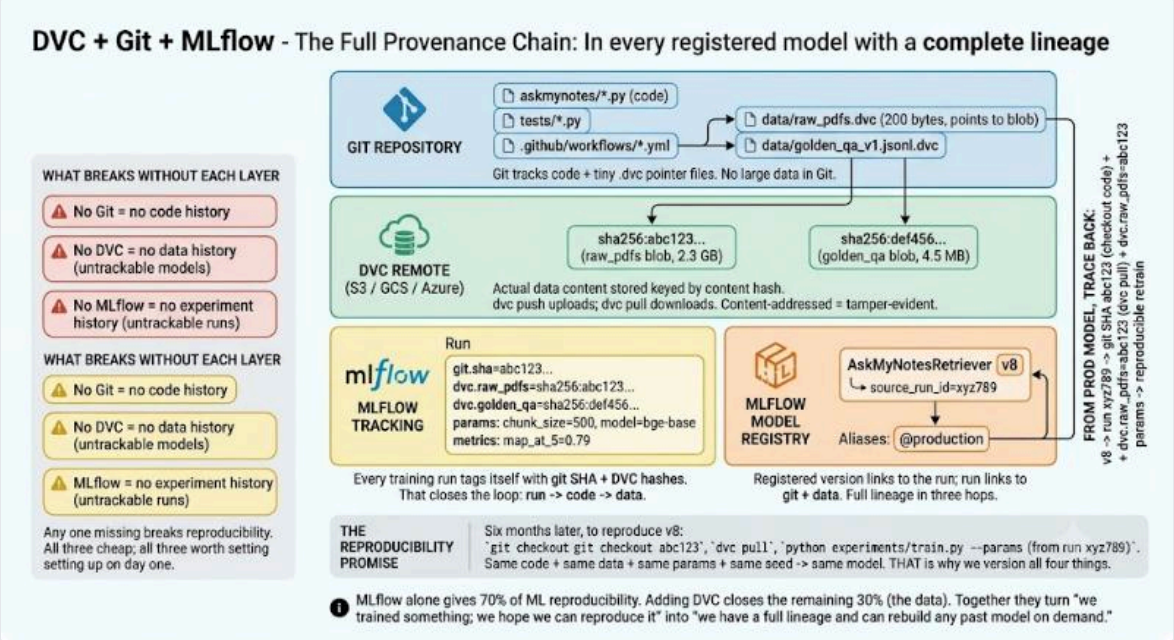
```
#  README.md                    badges + 12-command bring-up +
walkthrough
```

CI/CD FOR ML - DATA VERSIONING, MODEL CONTRACTS, AND THE THREE PIPELINES

Claude • AI Assistant



The Three Pipelines - Code, Training, Serving as Separate Systems



DVC + Git + MLflow - The Full Provenance Chain

yaml — Complete dvc.yaml for AskMyNotes' training pipeline with params.yaml

```yaml
# ---- dvc.yaml ----
# A lightweight DAG of stages, each with declared deps + outs + params.
# `dvc repro` re-runs only stages whose inputs or params changed.

stages:
  extract:
    cmd: python pipelines/extract_pdfs.py --in data/raw_pdfs --out data/text
    deps:
      - pipelines/extract_pdfs.py
      - data/raw_pdfs
    outs:
      - data/text

  chunk:
    cmd: python pipelines/chunk.py --in data/text --out data/chunks.jsonl
    deps:
      - pipelines/chunk.py
      - data/text
    params:
      - chunker.size
      - chunker.overlap
    outs:
      - data/chunks.jsonl

  embed:
    cmd: python pipelines/embed.py --in data/chunks.jsonl --out
data/embeddings
```

```
    deps:
      - pipelines/embed.py
      - data/chunks.jsonl
    params:
      - embedder.model
      - embedder.batch_size
    outs:
      - data/embeddings

  evaluate:
    cmd: python pipelines/evaluate.py --embeddings data/embeddings \
         --golden data/golden_qa_v1.jsonl --out metrics/eval.json
    deps:
      - pipelines/evaluate.py
      - data/embeddings
      - data/golden_qa_v1.jsonl
    metrics:
      - metrics/eval.json:
          cache: false

  register:
    cmd: python pipelines/register.py --run-id-file /tmp/run_id.txt
    deps:
      - pipelines/register.py
      - metrics/eval.json
    always_changed: true                # always run (registers to MLflow)


# ---- params.yaml ----
chunker:
  size: 500
  overlap: 50

embedder:
  model: BAAI/bge-base-en-v1.5
  batch_size: 32


# ---- Everyday commands ----
# dvc repro                             # run full pipeline; skip up-to-date
stages
# dvc repro evaluate                    # run just evaluate + any stale
predecessors
# dvc dag                               # ASCII DAG of stages
# dvc metrics show                      # show latest metrics from
metrics/eval.json
```

```
# dvc metrics diff HEAD~1          # compare vs previous commit
# dvc params diff                  # show what params changed
# dvc exp run                      # run as an EXPERIMENT (tracked
separately)
# dvc exp show                     # tabular view of experiments
# dvc push                         # push data + intermediate outputs to
remote
```

python — Contract test suite that gates promotion in CI - the ML unit tests

```python
# tests/contract/test_retriever_contract.py
# Run against @candidate before promoting to @staging.
# All tests must pass for the promotion script to advance the alias.

import json
import time
from pathlib import Path

import mlflow
import mlflow.pyfunc
import pytest
from askmynotes.data import load_golden_qa
from askmynotes.eval import compute_map_at_k, compute_recall_at_k

REGISTERED_MODEL = 'AskMyNotesRetriever'
CANDIDATE_URI = f'models:/{REGISTERED_MODEL}@candidate'
PREV_PROD_URI = f'models:/{REGISTERED_MODEL}@production'

# Contract thresholds - promoted models must beat these
MIN_MAP_AT_5 = 0.75
MIN_RECALL_AT_10 = 0.82
MAX_P95_LATENCY_MS = 200
MAX_FAIRNESS_GAP = 0.05
MAX_DRIFT_KS_STAT = 0.15


@pytest.fixture(scope='module')
def candidate():
    return mlflow.pyfunc.load_model(CANDIDATE_URI)


@pytest.fixture(scope='module')
def prev_production():
    try:
        return mlflow.pyfunc.load_model(PREV_PROD_URI)
```

```
    except mlflow.exceptions.RestException:
        pytest.skip('No previous @production version - skipping compat
check')


@pytest.fixture(scope='module')
def golden():
    return load_golden_qa()


# ---- 1. Schema check ----
def test_schema_accepts_expected_inputs(candidate, golden):
    sample = {'query': golden[0].question, 'candidates':
golden[0].candidates}
    result = candidate.predict(sample)
    assert isinstance(result, list)
    assert len(result) == len(sample['candidates'])
    assert all(isinstance(x, float) for x in result)
    assert all(-1.0 <= x <= 1.0 for x in result)     # cosine score range


# ---- 2. Offline quality checks ----
def test_meets_map_at_5_threshold(candidate, golden):
    map5 = compute_map_at_k(candidate, golden, k=5)
    assert map5 >= MIN_MAP_AT_5, f'MAP@5={map5:.3f} < required
{MIN_MAP_AT_5}'

def test_meets_recall_at_10_threshold(candidate, golden):
    r10 = compute_recall_at_k(candidate, golden, k=10)
    assert r10 >= MIN_RECALL_AT_10, f'Recall@10={r10:.3f} <
{MIN_RECALL_AT_10}'


# ---- 3. Latency budget ----
def test_latency_budget(candidate, golden):
    # Warm up JIT / caches
    for q in golden[:5]:
        candidate.predict({'query': q.question, 'candidates': q.candidates})
    latencies_ms = []
    for q in golden[:200]:
        t0 = time.perf_counter()
        candidate.predict({'query': q.question, 'candidates': q.candidates})
        latencies_ms.append((time.perf_counter() - t0) * 1000)
    p95 = sorted(latencies_ms)[int(len(latencies_ms) * 0.95)]
    assert p95 < MAX_P95_LATENCY_MS, f'p95 {p95:.1f}ms > budget
{MAX_P95_LATENCY_MS}ms'
```

```python
# ---- 4. Fairness slice ----
@pytest.mark.parametrize('group', ['academic', 'legal', 'medical'])
def test_meets_quality_per_group(candidate, golden, group):
    slice_ = [q for q in golden if q.group == group]
    if len(slice_) < 20:
        pytest.skip(f'Group {group} has fewer than 20 samples')
    map5 = compute_map_at_k(candidate, slice_, k=5)
    # Each group must ALSO meet the overall threshold - no silent
underperformance
    assert map5 >= MIN_MAP_AT_5 - MAX_FAIRNESS_GAP, \
        f'Group {group}: MAP@5={map5:.3f} below floor'


# ---- 5. Backwards compatibility ----
def test_output_shape_matches_prev_production(candidate, prev_production,
golden):
    sample = {'query': golden[0].question,
              'candidates': golden[0].candidates[:5]}
    new_out = candidate.predict(sample)
    old_out = prev_production.predict(sample)
    assert type(new_out) == type(old_out)
    assert len(new_out) == len(old_out)
    # Both must be lists of floats in the same range
    assert all(isinstance(x, float) for x in new_out)


# ---- 6. Wire into CI ----
# .github/workflows/train.yml step (runs after training):
#   - name: Contract tests against @candidate
#     env:
#       MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
#     run: pytest tests/contract/ -v --tb=short
#   - name: Promote @candidate -> @staging if contract tests pass
#     if: success()
#     run: python experiments/promote_best_model.py --target-alias staging
```

Student

Chapter 4 got AskMyNotes to alias-based promotion. What's still missing?

Claude • AI Assistant

Three things. **First**, we're versioning code and models but not the **data** those models were trained on.

A `.pkl` file in the registry with 'trained on August's Q&A set' as a free-text tag is not reproducible - August's data may not exist anywhere retrievable. **Second**, the promotion pipeline in 4.5 stopped at 'here's the winner; alias it'.

It didn't automate the **contract checks** (schema match, offline quality, latency budget, backwards-compat) that gate a real production ML promotion. **Third**, we don't yet have the notion of the **three pipelines** most production ML systems run in parallel - code pipeline (Chapter 3-4), training pipeline (new here), serving pipeline (Chapter 4.2).

This subsection covers **DVC** (Git-adjacent data versioning), the **model contract** checks that gate promotion in CI, and the **three-pipeline architecture** that separates code delivery from training delivery from serving delivery. Section 5.2 then covers how the training pipeline itself is orchestrated (Prefect / Metaflow / GitHub Actions).

Sections 5.3 and 5.4 build a full working example on AskMyNotes.

THE THREE PIPELINES

Student

What are these three pipelines?

Claude • AI Assistant

CONCEPT CARD

A common failure mode of early ML teams is trying to cram training, deployment, and code delivery into one CI pipeline. It works for the first model; it breaks by the fourth. Real production ML systems separate three pipelines:

1. Code pipeline (the CI you already know): triggered on every push / PR. Runs lint, type check, tests, build. Produces a validated Docker image. Cadence: per commit. Chapter 3 and 4.1-4.2 built this.

2. Training pipeline: triggered on new data, on a schedule, on manual dispatch, or on drift alerts.

Extracts data -> preprocesses -> trains -> evaluates -> registers a new model version if quality thresholds pass -> optionally promotes through aliases. Cadence: daily / weekly / on-demand.

Section 5.2 orchestrates this.

3. Serving pipeline (the CD you already know, model-aware): triggered on a new `@production` alias or a code change.

Rolls out the current `@production` model to canary -> full production. Cadence: per model promotion or per code deploy.

Chapter 4.2 and 4.5 covered the mechanics.

Why three, not one? Because they run on different clocks, need different resources (training needs GPUs; serving needs low-latency CPUs; code CI needs neither), have different failure modes, and are owned by different personas (data scientists own training, backend engineers own serving, everyone owns code). Coupling them means every training-data change waits for a code review; every code change re-triggers training.

Both are wrong.

DVC - GIT-ADJACENT DATA VERSIONING

Student

Why can't I just put data in Git?

Claude • AI Assistant

CONCEPT CARD

Because Git struggles at scale with binary files. A 500MB PDF corpus in Git bloats every clone; a 20GB training set is unusable.

DVC solves this by storing the data content in a **remote** (S3, GCS, Azure Blob, SSH, local NAS) and keeping only a tiny `.dvc` **pointer file** in Git. The `.dvc` file contains the content-hash and remote URL; the actual bytes live in the remote.

bash — Code Example

```
pip install 'dvc[s3]'                          # or [gs], [azure], [ssh]
dvc init                                       # creates .dvc/ (like git
init)
git add .dvc .dvcignore
git commit -m 'chore: initialize DVC'

# Configure a remote (where the actual data blobs live)
dvc remote add -d storage s3://my-bucket/askmynotes-data
git add .dvc/config && git commit -m 'chore: add S3 remote'
```

```
# Track a dataset
dvc add data/golden_qa_v1.jsonl              # creates
data/golden_qa_v1.jsonl.dvc
git add data/golden_qa_v1.jsonl.dvc .gitignore
git commit -m 'data: add golden QA v1 (1,247 questions, 5,832 chunks)'

# Push the actual data blob to the remote
dvc push

# Someone else clones the repo
git clone https://github.com/user/askmynotes.git
cd askmynotes
dvc pull                                     # download data blobs from
remote
```

The `.dvc` file (~200 bytes) travels in Git like any tracked file. Multiple versions of the dataset over time = multiple git commits changing the `.dvc` file. `git log data/golden_qa_v1.jsonl.dvc` shows the history of the dataset just like it shows the history of a source file.

WIRING DATA VERSION INTO MLFLOW

Student

How does DVC connect to my MLflow experiments?

Claude • AI Assistant

CONCEPT CARD

Log the DVC hash as a tag on every training run:

python — Code Example

```python
import subprocess, mlflow

def dvc_hash(path: str) -> str:
    # Read the .dvc file's md5 field, the content hash of the data blob
    with open(f'{path}.dvc') as f:
        for line in f:
            if line.strip().startswith('md5:'):
                return line.split(':', 1)[1].strip()
    return 'unknown'
```

```
with mlflow.start_run():
    mlflow.set_tag('dvc.golden_qa', dvc_hash('data/golden_qa_v1.jsonl'))
    mlflow.set_tag('dvc.raw_pdfs', dvc_hash('data/raw_pdfs'))
    mlflow.set_tag('git.sha', subprocess.check_output(['git', 'rev-parse',
'HEAD']).decode().strip())
    # ... train, log params, log metrics ...
```

Now every registered model version links back to: (a) the exact git SHA of the code, (b) the exact DVC hashes of every input dataset, (c) the params/metrics/artifacts of the run. **Six months later**, reproducing that model becomes: `git checkout <sha> && dvc pull && python experiments/train.py --params <logged params>` .

That's the reproducibility promise Section 4.4 talked about, actually made real.

DVC PIPELINES - THE `DVC.YAML` FILE

Student

DVC only stores data?

Claude • AI Assistant

CONCEPT CARD

Not quite. DVC also has a lightweight pipeline runner (`dvc.yaml`) - stages with inputs, outputs, and commands. When you run `dvc repro` , DVC re-executes only the stages whose inputs changed:

yaml — Code Example

```yaml
# dvc.yaml
stages:
  extract:
    cmd: python pipelines/extract_pdfs.py --out data/text
    deps:
      - pipelines/extract_pdfs.py
      - data/raw_pdfs
    outs:
      - data/text

  chunk:
    cmd: python pipelines/chunk.py --in data/text --out data/chunks.jsonl
    deps:
```

```
      - pipelines/chunk.py
      - data/text
    params:
      - chunker.size
      - chunker.overlap
    outs:
      - data/chunks.jsonl

  embed:
    cmd: python pipelines/embed.py --in data/chunks.jsonl --out
data/embeddings
    deps:
      - pipelines/embed.py
      - data/chunks.jsonl
    params:
      - embedder.model
    outs:
      - data/embeddings

  evaluate:
    cmd: python pipelines/evaluate.py --embeddings data/embeddings --golden
data/golden_qa_v1.jsonl
    deps:
      - pipelines/evaluate.py
      - data/embeddings
      - data/golden_qa_v1.jsonl
    metrics:
      - metrics/eval.json:
          cache: false
```

Params come from a `params.yaml` file that's committed alongside. **DVC computes a DAG** from `deps` / `outs`; running `dvc repro evaluate` walks the DAG and re-runs only the stages whose inputs or params changed since last run.

Great for local iteration without wasteful full re-runs.

For production-scale orchestration, DVC pipelines are usually a stepping stone - Section 5.2 covers heavier orchestrators (Prefect, Airflow, Metaflow) for scheduled runs across multiple environments. But for solo work + reproducibility, `dvc repro` alone earns its keep.

MODEL CONTRACTS - THE AUTOMATED GATES FOR PROMOTION

Student

You mentioned 'contract checks'. What are they?

Claude • AI Assistant

CONCEPT CARD

A **model contract** is the set of guarantees a model must satisfy to be promoted from `@candidate` to `@staging` (or `@staging` to `@production`). These are automated tests that run in CI against a registered model version, exactly like unit tests run against code.

Six common categories:

1. Schema check: does the model accept its declared input types/shapes and return its declared output shape? Uses the signature saved with the model.

2. Offline quality check: does the model beat a minimum metric threshold on a fixed **golden dataset**? Uses the same golden set every time so scores are comparable across versions.

3. Latency budget check: is p95 inference time under the configured limit? Runs on a fixed hardware runner for comparability.

4. Fairness slice check: is the metric gap between subgroups (e.g., accuracy on group A vs group B) within tolerance? Requires labeled slices in the golden dataset.

5. Drift-vs-baseline check: is the prediction distribution close to a reference baseline? Uses KS test, Population Stability Index (PSI), or similar. Catches subtle regressions that raw accuracy misses.

6. Backwards-compat check: does the new model's signature match the previous production version's? If the output shape or enum values change, downstream consumers break silently. This is where 'small model swap' becomes 'quiet outage'.

Each check is a pytest test file that loads the candidate model and asserts. The training/promotion pipeline runs the full contract test suite; only if all pass does it advance the alias. In practice, this is often called **model unit testing** or **CI for models**.

A CONCRETE CONTRACT TEST

Student

Show me one.

Claude • AI Assistant

EXAMPLE CARD

Contract test for AskMyNotesRetriever's schema and quality:

python — Code Example

```python
# tests/contract/test_retriever_contract.py
import pytest
import mlflow.pyfunc
import time
from askmynotes.data import load_golden_qa
from askmynotes.eval import compute_map_at_k

MODEL_URI = 'models:/AskMyNotesRetriever@candidate'
BASELINE_METRICS = 'baseline/retriever_baseline.json'
MIN_MAP_AT_5 = 0.75
MAX_P95_LATENCY_MS = 200
MAX_FAIRNESS_GAP = 0.05

@pytest.fixture(scope='module')
def model():
    return mlflow.pyfunc.load_model(MODEL_URI)

@pytest.fixture(scope='module')
def golden():
    return load_golden_qa()

def test_schema_matches_signature(model, golden):
    result = model.predict({'query': golden[0].question,
                            'candidates': golden[0].candidates})
    assert isinstance(result, list)
    assert all(isinstance(s, float) for s in result)
    assert len(result) == len(golden[0].candidates)

def test_meets_quality_threshold(model, golden):
    map5 = compute_map_at_k(model, golden, k=5)
    assert map5 >= MIN_MAP_AT_5, f'MAP@5={map5:.3f} < {MIN_MAP_AT_5}'

def test_latency_within_budget(model, golden):
    # Warm up
    for q in golden[:5]:
        model.predict({'query': q.question, 'candidates': q.candidates})
    latencies_ms = []
    for q in golden[:200]:
        t = time.perf_counter()
```

```
        model.predict({'query': q.question, 'candidates': q.candidates})
        latencies_ms.append((time.perf_counter() - t) * 1000)
    p95 = sorted(latencies_ms)[int(len(latencies_ms) * 0.95)]
    assert p95 < MAX_P95_LATENCY_MS, f'p95 latency {p95:.1f}ms >
{MAX_P95_LATENCY_MS}ms'

def test_fairness_gap_bounded(model, golden):
    map5_by_group = {
        g: compute_map_at_k(model, [q for q in golden if q.group == g], k=5)
        for g in {'academic', 'legal', 'medical'}
    }
    gap = max(map5_by_group.values()) - min(map5_by_group.values())
    assert gap < MAX_FAIRNESS_GAP, f'fairness gap {gap:.3f} >
{MAX_FAIRNESS_GAP}'

def test_backwards_compat_with_prev_production(model):
    prev =
mlflow.pyfunc.load_model('models:/AskMyNotesRetriever@production')
    sample = {'query': 'test', 'candidates': ['a', 'b', 'c']}
    new_out = model.predict(sample)
    old_out = prev.predict(sample)
    assert type(new_out) is type(old_out)
    assert len(new_out) == len(old_out)
```

Run this as a job in the training pipeline: only if ALL contract tests pass does the promotion script move `@candidate` to `@staging`. This is CI for models.

ANALOGY

Model contracts are **unit tests for models**. A code unit test says 'my function must return X for input Y'.

A model contract says 'my model must achieve MAP@5 >= 0.75 on the golden dataset, respond in under 200ms p95, and its output shape must match the previous production version'. Both are automated gates that block bad artifacts from reaching production; the shape of the assertion is different, the discipline is identical.

REAL WORLD APPLICATION

A team shipped a model with a subtly-changed output enum ('positive' -> 'POSITIVE'). Every downstream consumer that switched on the enum value silently misbehaved.

The fix took 30 seconds; the outage was 6 hours because the bad model was auto-promoted through canary before the downstream fallout showed up in metrics. After the incident: a backwards-compat contract test blocked such promotions.

Cost of the test: 5 lines. Cost of not having it: 6 hours + reputation.

Contract tests are cheap insurance.

⚠ WARNING

Do NOT let contract tests run against the LIVE production model. Load only the CANDIDATE version. A contract test that accidentally exercises production traffic is worse than no test - it's an incident.

COMMON PITFALL / MISTAKE

Skipping data versioning because 'we'll never need to reproduce that old model'. Six months later a regulator asks 'show us the training data for the model that made this decision on Sept 14'.

Without DVC-style hashing, you're guessing. Add DVC early; it costs nothing while nothing goes wrong.

EXAM TIP CARD

Interview question: 'How do you promote a model to production safely?' Complete answer covers: 'Register new version via MLflow, set `@candidate` alias, run a contract test suite (schema + offline quality + latency + fairness + drift + backwards-compat), promote `@candidate` to `@staging` on green, shadow deploy for a day, canary at 5% for hours, then `@production`. Every stage traceable back to git SHA and DVC data hash via MLflow tags.

Rollback is an alias flip.' That's Chapter 4 + 5 in one paragraph.

PRACTICE QUIZ

Your team versions code in Git and models in MLflow, but the training dataset lives on someone's laptop that later crashed. What's the concrete cost the next time you need to reproduce a 3-month-old model?

A) Nothing - retrain from newer data

B) The model is fundamentally unreproducible; you can rebuild an *approximate* replacement but never the exact model

C) MLflow will regenerate the dataset

D) Git will restore the dataset from history

▼ Reveal Correct Answer

Answer: B. Without versioned data (DVC or equivalent), the model is unreproducible. Auditability and rollback both suffer. DVC + MLflow together are what make ML reproducibility real.

EXERCISE

On AskMyNotes: (a) `pip install 'dvc[s3]'`, `dvc init`, and configure a local remote (`dvc remote add -d storage /tmp/dvc-remote`), (b) `dvc add data/raw_pdfs` and commit the resulting `.dvc` file, (c) log the DVC hash as `mlflow.set_tag('dvc.raw_pdfs', ...)` in your tracking script, (d) create `tests/contract/test_retriever_contract.py` with tests for schema, quality threshold, and latency, (e) add a `contract-tests` job to `.github/workflows/train.yml` that runs against `@candidate` before promoting to `@staging`.

AI DEEP DIVE PROMPT

Ask an AI: *'Design a full model contract test suite for a text-classification model. Include schema, offline quality, latency budget, fairness across three subgroups, drift-vs-baseline (KS test), and backwards-compatibility.

Structure as pytest with fixtures loading @candidate. Assume the model is registered in MLflow.'*

SUMMARY CARD

CI/CD for ML = three pipelines: **code pipeline** (CI, per commit), **training pipeline** (new data / schedule / drift, per day/week), **serving pipeline** (CD, per model promotion or code deploy). Each has its own trigger, cadence, resources, and owner.

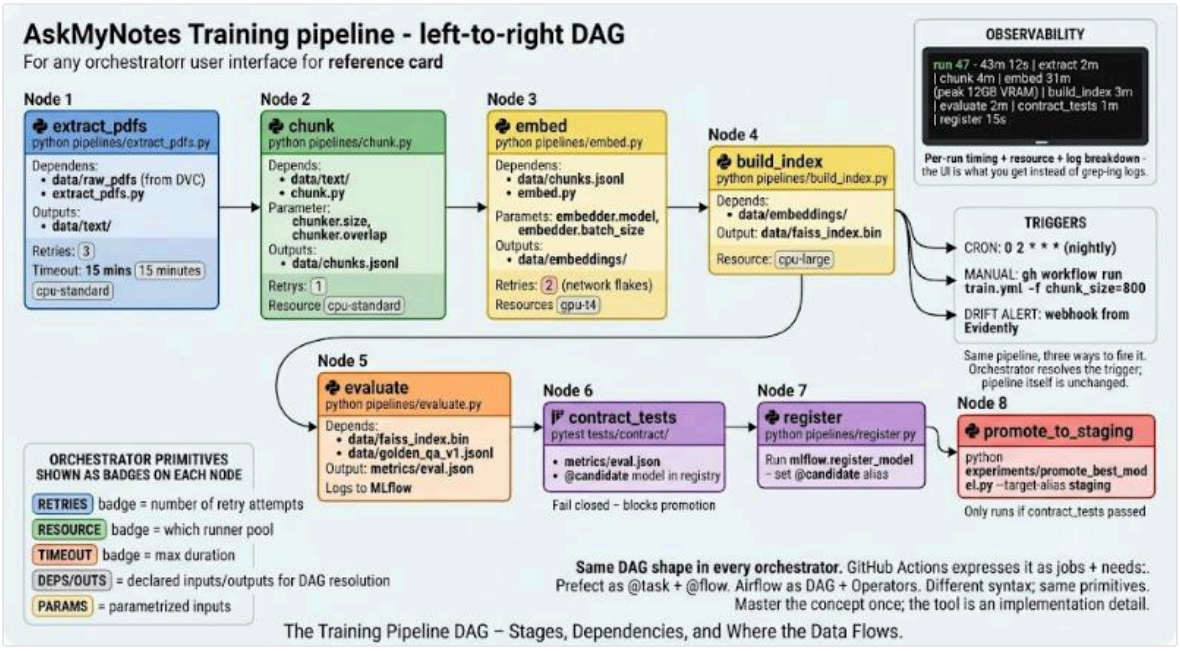
DVC versions large data using Git-adjacent `.dvc` pointer files with content stored in remote (S3/GCS/etc.); `dvc push` / `dvc pull` mirror `git push` / `git pull`. Log DVC hashes as MLflow tags for full lineage.

Model contracts are automated tests against `@candidate` before promotion: schema + offline quality + latency + fairness + drift + backwards-compat. Contract tests block bad models from reaching production - the ML equivalent of unit tests blocking bad code.

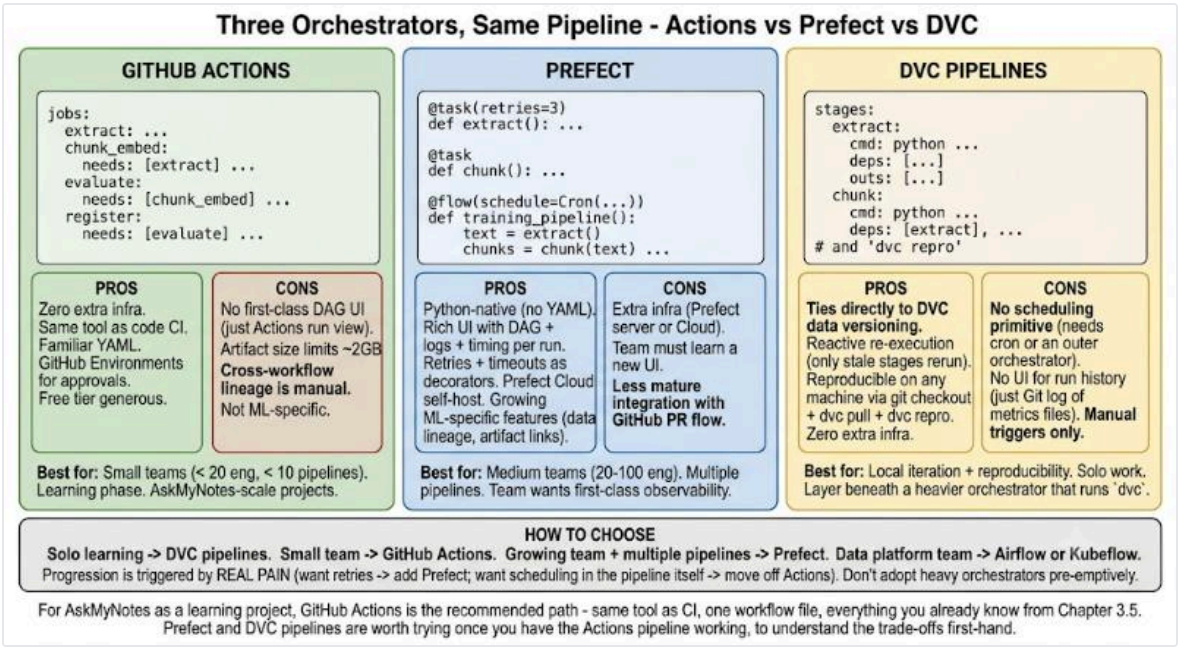
Section 5.2 orchestrates the training pipeline; 5.3-5.4 build the full end-to-end lab.

ML PIPELINE AUTOMATION - PREFECT, METAFLOW, AND GITHUB ACTIONS

Claude • AI Assistant



The Training Pipeline DAG - Stages, Dependencies, and Where the Data Flows



Three Orchestrators, Same Pipeline - Actions vs Prefect vs DVC

yaml — Complete .github/workflows/train.yml for AskMyNotes' training pipeline

```yaml
# .github/workflows/train.yml
# Nightly cron + manual dispatch. Runs extract -> embed -> evaluate ->
# contract tests -> register + promote. Uses matrix for embedding-model
sweep.

name: Training pipeline

on:
```

```yaml
  schedule:
    - cron: '0 2 * * *'                # nightly 02:00 UTC
  workflow_dispatch:
    inputs:
      chunk_size:
        description: 'Chunk size'
        default: '500'
        required: true
      embedders:
        description: 'Embedding models to try (comma-separated)'
        default: 'BAAI/bge-base-en-v1.5'
        required: true
      promote_to:
        description: 'Alias to promote winner to (candidate/staging)'
        default: 'candidate'
        type: choice
        options: ['candidate', 'staging']

concurrency:
  group: train-${{ github.ref }}
  cancel-in-progress: false            # don't interrupt an in-flight train

env:
  MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
  MLFLOW_TRACKING_TOKEN: ${{ secrets.MLFLOW_TOKEN }}
  AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
  AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

jobs:
  # ---- Stage 1: extract raw PDFs to text ----
  extract:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -e .[dev,dvc-s3]
      - name: Pull data from DVC
        run: dvc pull data/raw_pdfs data/golden_qa_v1.jsonl
      - name: Extract PDFs -> text
        run: python pipelines/extract_pdfs.py --in data/raw_pdfs --out
data/text
      - uses: actions/upload-artifact@v4
        with:
          name: text-and-golden
```

```
          path: |
            data/text/
            data/golden_qa_v1.jsonl
          retention-days: 7

  # ---- Stage 2: chunk + embed (matrix across embedding models) ----
  chunk_and_embed:
    needs: [extract]
    runs-on: [self-hosted, gpu]        # or 'ubuntu-latest' for CPU-only
    timeout-minutes: 60
    strategy:
      fail-fast: false
      matrix:
        embedder: ${{ fromJSON(format('["{0}"]',
github.event.inputs.embedders || 'BAAI/bge-base-en-v1.5')) }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -e .[dev]
      - uses: actions/download-artifact@v4
        with: { name: text-and-golden, path: data/ }
      - name: Chunk
        run: python pipelines/chunk.py --in data/text --out data/chunks.jsonl
                                       --size ${{ inputs.chunk_size || '500'
}}
      - name: Embed
        run: python pipelines/embed.py --in data/chunks.jsonl
                                       --model ${{ matrix.embedder }}
                                       --out data/embeddings
      - name: Build index
        run: python pipelines/build_index.py --in data/embeddings --out
data/index.bin
      - name: Evaluate + log to MLflow
        id: eval
        run: |
          python pipelines/evaluate_and_register.py \
            --index data/index.bin \
            --golden data/golden_qa_v1.jsonl \
            --embedder "${{ matrix.embedder }}" \
            --chunk-size ${{ inputs.chunk_size || '500' }} \
            --output run_id.txt
          echo "run_id=$(cat run_id.txt)" >> $GITHUB_OUTPUT

  # ---- Stage 3: contract tests against @candidate, then promote ----
  contract_and_promote:
```

```
    needs: [chunk_and_embed]
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -e .[dev]
      - name: Contract tests against @candidate
        run: pytest tests/contract/ -v --tb=short
      - name: Promote @candidate -> @${{ inputs.promote_to || 'candidate' }}
        if: success() && inputs.promote_to == 'staging'
        run: python experiments/promote_best_model.py --target-alias staging
      - name: Post summary
        if: always()
        run: |
          echo "### Training pipeline summary" >> $GITHUB_STEP_SUMMARY
          echo "- Chunk size: ${{ inputs.chunk_size || '500' }}" >>
$GITHUB_STEP_SUMMARY
          echo "- Embedder: ${{ inputs.embedders || 'default' }}" >>
$GITHUB_STEP_SUMMARY
          echo "- MLflow: ${{ env.MLFLOW_TRACKING_URI }}" >>
$GITHUB_STEP_SUMMARY
```

Student

Section 5.1 said the training pipeline runs on its own clock. What software actually runs it?

Claude • AI Assistant

An **orchestrator**. Where the code pipeline uses GitHub Actions and the serving pipeline uses whatever your production runtime is (Kubernetes, ECS, Cloud Run), the training pipeline typically uses one of: **GitHub Actions** (simplest, if you're already on GitHub), **Prefect** (Python-native, friendly), **Airflow** (heavy but mature), **Metaflow** (ML-first, Netflix-born), or **Kubeflow Pipelines** (Kubernetes-native, large-org).

This subsection covers the mental model of pipeline orchestration, shows the same pipeline in three tools (GitHub Actions, Prefect, DVC), and gives you the criteria for choosing among them. Section 5.3 wires up the full working example on AskMyNotes.

WHAT AN ORCHESTRATOR GIVES YOU

Student

Why not just run a Python script from cron?

Claude • AI Assistant

CONCEPT CARD

Cron + Python is fine for a single step that either succeeds or fails. An orchestrator earns its keep once you need any of these:

Multi-step DAG: 'extract -> preprocess -> train -> evaluate -> register -> deploy' where each step is independent, each has inputs and outputs, and some can parallelize.

Retry semantics: 'if the train step fails due to a transient GPU issue, retry 3 times with exponential backoff before failing the pipeline'.

Conditional branches: 'if eval metrics beat baseline, register the model; else fail the pipeline and alert'.

Parametrization + versioning: 'run the same pipeline with `chunk_size=800` and `embedder=text-embedding-3-small` ; keep the results comparable to the previous run'.

Observability: a UI showing 'this pipeline run took 43 minutes; the embed step took 31 of those and used 12GB VRAM peak; here's the log from each step'.

Trigger management: cron, event-based, drift-alert, manual dispatch - all resolved to the same run object.

Retries + backfills: 'the pipeline failed for Aug 12-15; rerun those four days without touching the others'.

None of these are impossible with cron + bash + care. All are error-prone. An orchestrator collapses hundreds of lines of glue into a decorator or a YAML block.

THE DAG MENTAL MODEL

Student

What does an ML pipeline actually look like?

Claude • AI Assistant

CONCEPT CARD

A **directed acyclic graph** (DAG) of stages. Each stage has:

- **Inputs**: files or upstream stages' outputs
- **Command / function**: the actual work to run
- **Outputs**: files or artifacts that downstream stages depend on
- **Retries / timeout / resources**: how to handle failure and where to run

For AskMyNotes, a typical training pipeline:

code — Code Example

```
extract_pdfs -> chunk -> embed -> build_index
                                \
                                 -> evaluate -> contract_tests -> register ->
promote
```

`extract_pdfs` reads raw PDFs and produces plain-text files. `chunk` splits text into overlapping chunks.

`embed` turns chunks into vectors. `build_index` writes a FAISS index.

`evaluate` runs the golden Q&A benchmark and produces metrics. `contract_tests` runs the Section 5.1 contract suite.

`register` calls `mlflow.register_model`. `promote` moves the alias.

The orchestrator's job: execute this DAG, respect dependencies (don't `evaluate` before `build_index`), parallelize what can parallelize (embed + build_index parallelize with independent chunks), handle failures, and record the run for later inspection.

GITHUB ACTIONS AS AN ORCHESTRATOR

Student

Can GitHub Actions run the training pipeline?

Claude • AI Assistant

CONCEPT CARD

Yes, and for small teams it's often the pragmatic choice - one tool for both code CI and training pipelines. Multi-step ML pipelines map onto GitHub Actions jobs + `needs:` sequencing (from Section 3.5):

yaml — Code Example

```yaml
# .github/workflows/train.yml (excerpt)
name: Training pipeline

on:
  schedule: [{ cron: '0 2 * * *' }]     # nightly at 02:00 UTC
  workflow_dispatch:
    inputs:
      chunk_size: { default: '500', required: true }
      embedder:   { default: 'BAAI/bge-base-en-v1.5', required: true }

jobs:
  extract:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Pull DVC data
        run: dvc pull data/raw_pdfs
      - name: Extract PDFs
        run: python pipelines/extract_pdfs.py --in data/raw_pdfs --out
data/text
      - uses: actions/upload-artifact@v4
        with: { name: text, path: data/text }

  chunk_and_embed:
    needs: [extract]
    runs-on: [self-hosted, gpu]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with: { name: text, path: data/text }
      - name: Chunk + Embed
        run: |
          python pipelines/chunk.py --in data/text --out data/chunks.jsonl
          python pipelines/embed.py --in data/chunks.jsonl --out
data/embeddings
      - uses: actions/upload-artifact@v4
        with: { name: embeddings, path: data/embeddings }

  evaluate_and_register:
    needs: [chunk_and_embed]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
```

```
      with: { name: embeddings, path: data/embeddings }
    - name: Evaluate + track to MLflow
      env: { MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }} }
      run: python pipelines/evaluate_and_register.py
    - name: Contract tests
      run: pytest tests/contract/ -v
    - name: Promote @candidate -> @staging
      if: success()
      run: python experiments/promote_best_model.py --target-alias staging
```

Simple, one config file, uses everything you already know from Chapter 3.5. Downsides: no first-class DAG UI (just the Actions run view), no cross-workflow lineage, and artifact size limits (~2GB per artifact).

For AskMyNotes-scale, entirely sufficient. For 20+ pipelines with complex retries, graduate to Prefect or Airflow.

PREFECT - PYTHON-NATIVE FLOWS

Student

What does Prefect look like?

Claude • AI Assistant

EXAMPLE CARD

The same pipeline as a Prefect flow:

python — Code Example

```python
# pipelines/prefect_training.py
from prefect import flow, task
from prefect.schedules import CronSchedule
import mlflow

@task(retries=3, retry_delay_seconds=60)
def extract(raw_dir: str) -> str:
    from askmynotes.extract import extract_all
    return extract_all(raw_dir, out='data/text')

@task
def chunk(text_dir: str, size: int, overlap: int) -> str:
    from askmynotes.chunker import chunk_dir
```

```
    return chunk_dir(text_dir, size=size, overlap=overlap,
out='data/chunks.jsonl')

@task(retries=2)
def embed(chunks_path: str, model: str) -> str:
    from askmynotes.embedder import embed_file
    return embed_file(chunks_path, model=model, out='data/embeddings')

@task
def evaluate_and_register(embeddings_path: str, params: dict) -> str:
    from askmynotes.eval import full_eval
    with mlflow.start_run() as run:
        mlflow.log_params(params)
        metrics = full_eval(embeddings_path)
        mlflow.log_metrics(metrics)
        if metrics['map_at_5'] >= 0.75:
            mv = mlflow.register_model(
                model_uri=f'runs:/{run.info.run_id}/model',
                name='AskMyNotesRetriever',
            )
            return mv.version
    return None

@flow(name='askmynotes-training',
      schedule=CronSchedule(cron='0 2 * * *', timezone='UTC'))
def training_pipeline(chunk_size: int = 500, overlap: int = 50,
                      model: str = 'BAAI/bge-base-en-v1.5'):
    text_dir = extract(raw_dir='data/raw_pdfs')
    chunks = chunk(text_dir, size=chunk_size, overlap=overlap)
    embeddings = embed(chunks, model=model)
    version = evaluate_and_register(embeddings,
                                    params={'chunk_size': chunk_size,
                                            'overlap': overlap,
                                            'embedder': model})
    return version

if __name__ == '__main__':
    training_pipeline()
```

Deploy to Prefect Cloud or self-hosted Prefect: `prefect deploy pipelines/prefect_training.py:training_pipeline` . Now the flow runs on schedule, has a UI showing every run's stages + logs + timing, retries transient failures automatically, and can be triggered by webhook / manual UI click / another flow.

Prefect vs Actions for training: Prefect gives you cleaner Python semantics, better observability, and first-class ML-oriented features (upstream data change triggers, per-task resource declarations). Actions gives you zero extra infra + the same tool as CI.

For AskMyNotes as a learning project: Actions is the recommended path; Prefect is worth trying once you have a working Actions pipeline and want the UI + retry semantics.

DVC PIPELINES AS A LIGHTWEIGHT MIDDLE GROUND

Student

What about DVC's own pipeline runner?

Claude • AI Assistant

CONCEPT CARD

For LOCAL reproducibility, `dvc.yaml` (Section 5.1) is often enough. `dvc repro` walks the DAG, skips stages whose inputs are up to date, and reruns only what changed - the same reactive re-execution logic as Make but tied into DVC's data versioning:

bash — Code Example

```
dvc repro                          # runs full pipeline; skips unchanged stages
dvc repro evaluate                 # runs evaluate + any stale predecessors
dvc dag                            # prints ASCII DAG
dvc metrics show                   # latest metrics/eval.json values
dvc metrics diff HEAD~1            # compare vs previous commit
dvc exp run --set-param chunker.size=800   # run as tracked experiment
dvc exp show                       # table of all experiments
```

When to use DVC pipelines: local iteration, reproducibility on any machine (checkout + dvc pull + dvc repro), simple projects with a single flow. **When to graduate**: cross-machine scheduling, complex retries, real-time triggers, multi-tenant runs - reach for GitHub Actions, Prefect, or Airflow.

THE SCHEDULED RETRAINING LOOP

Student

How does the training pipeline connect to drift and retraining?

Claude • AI Assistant

CONCEPT CARD

The end-state of MLOps maturity Level 2 (Section 4.3): a **retraining loop** that closes automatically when data drifts.

The loop:

1. Production serving logs every prediction (input + output) to a store.
2. A **drift monitor** (Evidently, Arize, custom) periodically compares recent production inputs to a reference distribution.
3. When drift score exceeds a threshold, the monitor triggers the training pipeline.
4. Training pipeline runs (extract -> chunk -> embed -> evaluate -> register).
5. Contract tests gate promotion; if pass, `@staging` alias moves to the new version.
6. Shadow deploy + canary + full promotion (Sections 4.5).
7. Drift monitor's reference distribution updated to new production baseline.

For AskMyNotes: 'new domain of PDFs starts arriving (medical, say, when the training data was all academic) -> input drift detected -> retraining pipeline fires -> new retriever trained on augmented corpus -> contract tests confirm no regression on old academic queries + improvement on medical queries -> alias promoted -> production seamlessly handles the new domain'. That's the full MLOps promise.

Sections 5.3-5.4 build a simplified version end-to-end.

ANALOGY

An orchestrator is **the operating system for your training pipeline**. Your training scripts are the applications; the orchestrator handles scheduling, resource allocation, retries on failure, and provides the UI you look at to understand what's running.

Without it, every pipeline is a bespoke bash script; with it, all your pipelines follow the same shape and are managed the same way.

REAL WORLD APPLICATION

A common team progression: 'we run training as a manual Jupyter notebook' -> 'we run it as a Python script from cron' -> 'we run it as a GitHub Actions workflow with proper stages' -> 'we move to Prefect once we have 5+ pipelines and want the UI' -> 'we adopt Airflow / Kubeflow when we have a data platform team owning the scheduling infra'. Each step is triggered by real pain, not by wanting to check a box.

Most small ML teams (< 20 engineers, < 10 models) live happily at the GitHub Actions or Prefect step for years.

WARNING

Do NOT let orchestrator secrets leak into training scripts. Store `MLFLOW_TRACKING_URI`, DVC remote credentials, GPU cluster tokens, and cloud storage keys in the orchestrator's secret store (GitHub Actions secrets, Prefect Blocks, Airflow Connections).

Training code should read them from environment variables set by the orchestrator - never hardcode.

COMMON PITFALL / MISTAKE

Combining code CI and training pipeline in one workflow. Every merged PR retrains the model - burns compute + GPU minutes + noise. Keep them separate: code CI runs per PR (fast); training pipeline runs on its own trigger (schedule / manual / drift).

EXAM TIP CARD

Interview question: 'How would you orchestrate a training pipeline?' Structured answer: 'For a small team on GitHub, I'd start with a GitHub Actions workflow with jobs for extract, embed, evaluate, register - using `needs:` for sequencing and artifacts to pass data. When we hit ~5 pipelines, I'd move to Prefect for better observability and retries.

For each stage I'd declare inputs/outputs, retries, timeout, and resource class. Triggers include cron for scheduled retrains, workflow*dispatch for manual, and repository*dispatch or webhook from a drift monitor.

The pipeline logs to MLflow, registers the model on eval-pass, and hands off to the promotion script from Section 4.5.'

PRACTICE QUIZ

Your training pipeline runs on cron every night, taking 45 minutes. On Monday it fails because a transient S3 outage prevented dvc pull. What's the cleanest fix?

A) Manually rerun the pipeline Tuesday morning

B) Wrap the dvc pull step with retries: 3 attempts with exponential backoff

C) Move to a different cloud provider

D) Disable the pipeline

▼ Reveal Correct Answer

*Answer: B. Transient failures are what orchestrator retry primitives exist for.

Every step of a training pipeline should declare retries for the kinds of failures that are actually transient (network, quota, GPU unavailable). Manual intervention (A) doesn't scale; C and D are non-solutions.*

EXERCISE

On AskMyNotes: (a) create `.github/workflows/train.yml` with jobs for extract, chunk*embed, evaluate*and*register - each depending on the previous via `needs:`, (b) add a schedule trigger (nightly cron) and a workflow*dispatch trigger with `chunk_size` and `embedder` inputs, (c) upload/download data between jobs via `actions/upload-artifact`, (d) integrate the MLflow tracking call from Section 4.4 + the promote script from Section 4.5, (e) trigger a manual run via `gh workflow run train.yml -f chunk_size=800`, watch it run in the Actions tab, and confirm a new `AskMyNotesRetriever` version appears in your MLflow registry.

AI DEEP DIVE PROMPT

Ask an AI: *'Convert my GitHub Actions training workflow to a Prefect flow with the same stages (extract, chunk, embed, evaluate, register). Include retries on the network-heavy stages, a cron schedule, and a manual-dispatch parameter for chunk_size.

Explain what Prefect gives me that Actions doesn't for this use case.'*

SUMMARY CARD

Orchestrators run training pipelines. Choose from **GitHub Actions** (simplest, fine for small teams), **Prefect** (Python-native, friendly UI), **Airflow** (mature, heavy), **Metaflow** (ML-first, Netflix-born), or **Kubeflow** (K8s-native, large orgs).

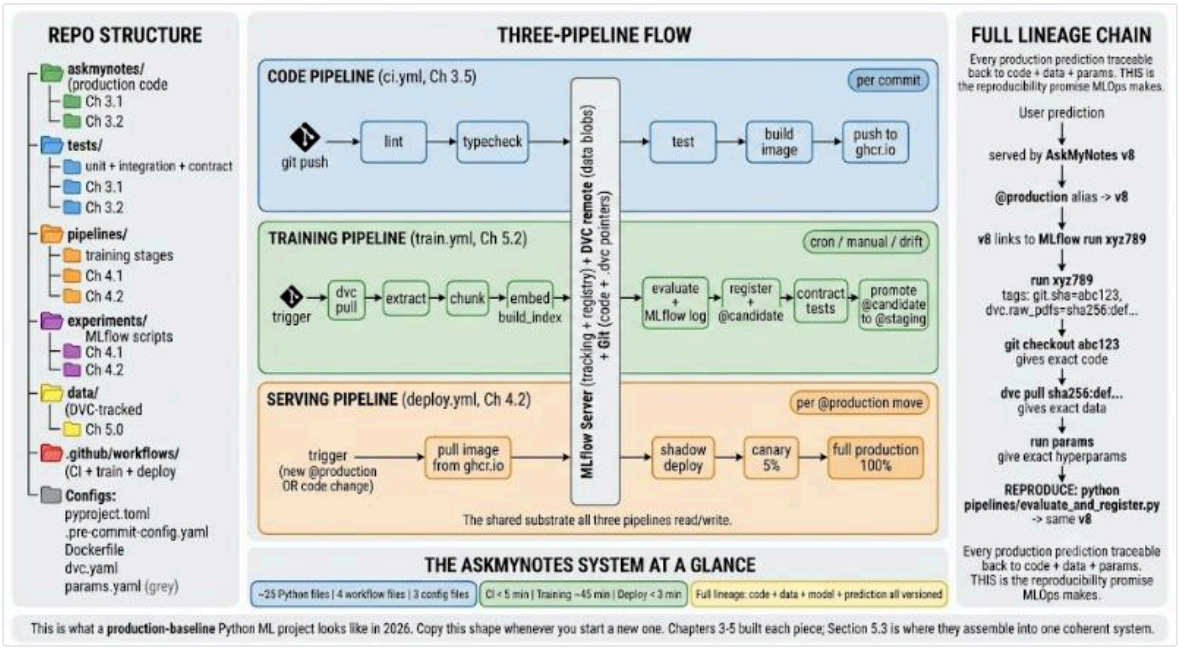
All model the pipeline as a **DAG** of stages with inputs, outputs, retries, and resources. **DVC pipelines** (`dvc.yaml` + `dvc repro`) are a lightweight middle ground for local reproducibility.

Triggers include **schedule (cron), data change, manual dispatch, drift alert, upstream job completion**. The mature loop is: production logs -> drift monitor -> triggers retraining -> new model registered -> contract tests -> alias promotion -> serving updates - all automated.

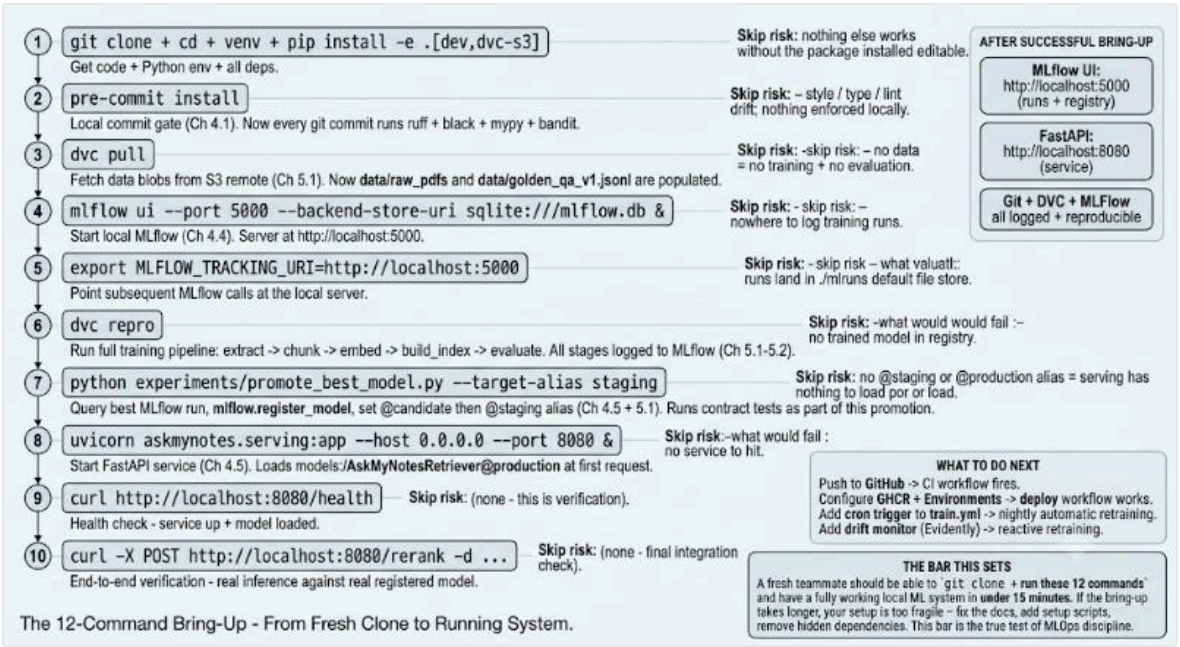
Keep training pipelines SEPARATE from code CI (different triggers, different resources). Sections 5.3-5.4 build a working example end-to-end.

FULL PIPELINE LAB - END-TO-END WALKTHROUGH ON ASKMYNOTES

Claude • AI Assistant



AskMyNotes Full System - Every File, Every Pipeline, Every Flow



The 12-Command Bring-Up - From Fresh Clone to Running System

dockerfile – Complete Dockerfile for AskMyNotes - multi-stage, cached, production-ready

```
# Dockerfile
# Multi-stage build: keep the runtime image small by discarding build tooling.
# Result: ~250MB image (base python + your deps + your app code + model loader).

# ---- Stage 1: builder - install deps into a wheelhouse ----
```

```
FROM python:3.12-slim AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /build

# Install build-only OS deps (compilers, ssl headers)
RUN apt-get update && apt-get install -y --no-install-recommends \
        build-essential \
        git \
    && rm -rf /var/lib/apt/lists/*

# Copy dep manifests first for maximum layer caching
COPY pyproject.toml README.md ./
COPY askmynotes/__init__.py askmynotes/

# Build a wheel of everything
RUN pip install --upgrade pip build && \
    pip wheel --wheel-dir=/wheels .


# ---- Stage 2: runtime - slim image with only what's needed at runtime ----
FROM python:3.12-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8080 \
    MODEL_NAME=AskMyNotesRetriever \
    MODEL_ALIAS=production

# Runtime-only OS deps (SSL, tzdata)
RUN apt-get update && apt-get install -y --no-install-recommends \
        ca-certificates \
        tzdata \
    && rm -rf /var/lib/apt/lists/*

# Non-root user for the runtime
RUN groupadd -r app && useradd -r -g app -d /app -s /sbin/nologin app
WORKDIR /app

# Install wheels from builder + the app source
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir --no-index --find-links=/wheels /wheels/*.whl
```

```
&& \
    rm -rf /wheels

COPY --chown=app:app askmynotes/ askmynotes/

USER app

# Healthcheck for orchestrators (K8s, Docker Compose, ECS)
HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:' + '${PORT:-8080}' +
'/health').read()" || exit 1

EXPOSE 8080

# Uvicorn worker for FastAPI. --workers 2 for prod concurrency;
# use gunicorn+uvicorn if you need more control over process management.
CMD ["sh", "-c", "uvicorn askmynotes.serving:app --host 0.0.0.0 --port
${PORT} --workers 2"]


# ---- Build + push ----
# docker build -t ghcr.io/<user>/askmynotes:sha-$(git rev-parse HEAD) .
# docker push ghcr.io/<user>/askmynotes:sha-$(git rev-parse HEAD)
# In deploy.yml (Ch 4.2), this is the exact command the docker/build-push-
action runs.
```

Student

I've got most of the pieces from Chapters 3 and 4. What does the finished AskMyNotes look like as one system?

Claude • AI Assistant

This subsection is the **assembly diagram** - the walkthrough of AskMyNotes as a finished MLOps-baseline project after Chapters 1-5. Every file, workflow, and dependency shown here is something you've built (or been shown how to build) in previous sections; here it comes together.

The lab is prescriptive: exact repo layout, exact command sequence to bring up all three pipelines from scratch, exact URLs and file paths. Do this hands-on before Section 5.4's MLflow+CI integration lab and Section 5.5's consolidation.

THE FINAL REPO SHAPE

Student

Show me the whole layout.

Claude • AI Assistant

CONCEPT CARD

After Chapters 1-5, AskMyNotes looks like this:

code — Code Example

```
askmynotes/                              # Python package (production
code)
  __init__.py
  chunker.py                            # 3.1 - core logic + type hints
  loader.py                             # PDF -> text
  embedder.py                           # text -> vectors
  vector_store.py                       # FAISS-backed retrieval
  retrieval.py                          # top-K search + reranker
  eval.py                               # MAP@K, MRR, per-slice metrics
  serving.py                            # 4.5 - FastAPI loading by alias
  data.py                               # golden Q&A loader

tests/                                  # 3.1-3.3 - unit + integration
  conftest.py                           # shared fixtures
  test_chunker.py                       # parametrized
  test_loader.py
  test_retrieval.py                     # mocked LLM
  integration/
    test_end_to_end.py                  # real PDF + real answer
  contract/                             # 5.1 - contract tests
    test_retriever_contract.py          # schema + quality + latency +
fairness

pipelines/                              # 5.2 - training pipeline stages
  extract_pdfs.py                       # stage 1: pdf -> text
  chunk.py                              # stage 2: text -> chunks
  embed.py                              # stage 3: chunks -> vectors
  build_index.py                        # stage 4: vectors -> index
  evaluate.py                           # stage 5: index + golden ->
metrics
```

```
  evaluate_and_register.py          # combined stage: eval + MLflow
log + register

experiments/                        # 4.4-4.5 + 5.1 - MLflow scripts
  track_run.py                      # 4.4 - manual tracked run
  find_best_run.py                  # 4.4 - query best from a sweep
  promote_best_model.py             # 4.5 - register + alias with
contract logic

data/                               # 5.1 - DVC-tracked datasets
  raw_pdfs/                         # (content in DVC remote, .dvc
file in Git)
  raw_pdfs.dvc
  golden_qa_v1.jsonl                # golden eval set (via DVC)
  golden_qa_v1.jsonl.dvc
  chunks.jsonl                      # produced by chunk stage
(dvc.yaml out)
  embeddings/                       # produced by embed stage
  index.bin                         # produced by build_index stage
  text/                             # produced by extract stage

metrics/                            # 5.1 - dvc.yaml metrics outputs
  eval.json                         # DVC-tracked JSON metrics

baseline/                           # 5.1 - reference metrics for
contract tests
  retriever_baseline.json           # thresholds + baseline
distribution

.github/workflows/                  # 3.5-5.2 - CI + train + deploy
  ci.yml                            # 3.5 - lint + typecheck + test
  quality.yml (or job in ci.yml)    # 4.1 - pre-commit gate
  deploy.yml                        # 4.2 - build + push + staging +
prod
  train.yml                         # 5.2 - training pipeline

dvc.yaml                            # 5.1 - DVC pipeline definition
params.yaml                         # 5.1 - pipeline params
(chunker.size, ...)
.dvc/                               # DVC config + cache
.dvcignore

.pre-commit-config.yaml             # 4.1 - hooks (ruff, black, mypy,
bandit)
pyproject.toml                      # tool configs + dev deps +
pytest/coverage
```

```
Dockerfile                          # 4.2 - the container image
.dockerignore
README.md                           # 3.5 - CI + coverage badges +
quickstart
.gitignore
```

About 25 Python files, 4 workflow files, 3 config files. This is the **shape of a production-baseline ML Python project** in 2026 - copy it whenever you start a new one.

THE BRING-UP SEQUENCE (FROM EMPTY FOLDER TO RUNNING)

Student

If I clone the repo fresh, what commands bring the whole system up?

Claude • AI Assistant

EXAMPLE CARD

Twelve commands total:

bash — Code Example

```bash
# ---- Fresh clone + local environment ----
git clone https://github.com/<you>/askmynotes.git
cd askmynotes
python -m venv .venv && source .venv/bin/activate
pip install -e .[dev,dvc-s3]              # install package + dev deps
pre-commit install                        # 4.1 - local commit gate

# ---- Pull versioned data (Section 5.1) ----
dvc pull                                  # fetch raw_pdfs + golden_qa
from remote

# ---- Start the MLflow tracking server locally ----
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db &
export MLFLOW_TRACKING_URI=http://localhost:5000

# ---- Run the training pipeline once locally (via DVC) ----
dvc repro                                 # runs extract -> chunk -> embed
-> evaluate

# ---- Register + promote the trained model ----
python experiments/promote_best_model.py --target-alias staging
```

```
# ---- Start the serving app locally ----
uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080 &

# ---- Verify end-to-end ----
curl http://localhost:8080/health
curl -X POST http://localhost:8080/rerank \
     -H 'Content-Type: application/json' \
     -d '{"query": "what is chunking?", "candidates": ["a", "b", "c"]}'
```

That's the entire local bring-up. On a fresh clone, all twelve commands should succeed - the pre-commit hooks, DVC data pull, MLflow tracking, DVC pipeline execution, MLflow registry promotion, and FastAPI serving all working together.

If any command fails, the failure IS the exercise - fix the missing piece and re-run.

THE ONE-DIAGRAM DATA + CONTROL FLOW

Student

How do all the pieces move data at runtime?

Claude • AI Assistant

CONCEPT CARD

Two flows overlaid on the same components:

Training flow (fires on cron / manual / drift):

1. GitHub Actions `train.yml` starts on trigger.
2. `actions/checkout` pulls latest code from Git.
3. `dvc pull data/raw_pdfs data/golden_qa_v1.jsonl` fetches data from S3 into the runner.
4. `python pipelines/extract_pdfs.py` -> `data/text/` .
5. `python pipelines/chunk.py` -> `data/chunks.jsonl` .
6. `python pipelines/embed.py` -> `data/embeddings/` .
7. `python pipelines/build_index.py` -> `data/index.bin` .
8. `python pipelines/evaluate_and_register.py` :

- Wraps in `with mlflow.start_run():`
- Logs params, metrics, artifacts, DVC hashes, git SHA
- If `map_at_5 >= threshold` , calls `mlflow.register_model` -> new version v_N

- Sets `@candidate` alias to v_N

13. `pytest tests/contract/ -v` - runs contract tests against `models:/AskMyNotesRetriever@candidate`.
14. If contracts pass: `python experiments/promote_best_model.py --target-alias staging` moves `@candidate -> @staging`.

Serving flow (per user request):

1. User sends `POST /rerank` to the FastAPI service.
2. `askmynotes.serving.load_model()` returns the cached model (loaded at first request from `models:/AskMyNotesRetriever@production`).
3. Model runs prediction.
4. FastAPI returns `{scores: [...], model_alias: 'production'}`.

Promotion flow (when @staging is trusted -> @production):

1. Human (or automated shadow/canary metric evaluator) decides `@staging` is safe.
2. `MlflowClient().set_registered_model_alias('AskMyNotesRetriever', 'production', v_N)` moves the alias.
3. Rolling restart of serving pods (Kubernetes rollout / ECS force-new-deployment / etc.).
4. Next request loads v_N via `@production`.
5. Application code unchanged; deploy done.

That's the entire system. Every arrow is a line of code you've already seen. Section 5.4 walks through implementing one round of this end-to-end.

THE CRITICAL FILES - WHAT EACH DOES IN ~10 SECONDS

Student

Give me the one-liner for each critical file.

Claude • AI Assistant

CONCEPT CARD

- **`pyproject.toml`**: tool configs (ruff, black, mypy, pytest, coverage), dev deps, package metadata. One file for the whole toolchain.
- **`.pre-commit-config.yaml`**: hooks that fire on every `git commit` - ruff + black + mypy + bandit + hygiene.
- **`Dockerfile`**: multi-stage build; base -> deps -> app; produces `ghcr.io/<user>/askmynotes:sha-...`.

- **`dvc.yaml` + `params.yaml`**: DVC pipeline DAG + parameters. `dvc repro` walks and executes.
- **`.github/workflows/ci.yml`**: on push/PR - lint + typecheck + tests + coverage. Blocks bad merges.
- **`.github/workflows/train.yml`**: on cron/manual/drift - runs training pipeline stages + contract tests + registry promotion.
- **`.github/workflows/deploy.yml`**: on main-branch push (post-CI) - build once, push to ghcr.io, deploy to staging, gated prod.
- **`askmynotes/serving.py`**: FastAPI service. `/health` + `/rerank`. Loads `models:/AskMyNotesRetriever@production` once at first request.
- *`*pipelines/*.py`**: modular stages (extract, chunk, embed, evaluate). Each is a Python script with argparse; all orchestrators call them the same way.
- **`experiments/promote_best_model.py`**: reads MLflow, finds winner, registers, sets alias with contract-gated logic.
- *`*tests/contract/*.py **: pytest against @candidate` for schema + quality + latency + fairness + backwards-compat.

THE FULL DATA LINEAGE FOR ONE PRODUCTION PREDICTION

Student

When a user request comes in, what's the full lineage of the model answering it?

Claude • AI Assistant

CONCEPT CARD

Trace it:

1. User request lands at FastAPI.
2. Model was loaded from `models:/AskMyNotesRetriever@production`.
3. `@production` alias resolves to `AskMyNotesRetriever v8` (say).
4. v8 links to MLflow run `xyz789` (via `source_run_id` tag).
5. Run `xyz789` has tags `git.sha=abc123` and `dvc.raw_pdfs=sha256:def...`.
6. `git checkout abc123` gives you the exact code that trained v8.
7. `dvc pull data/raw_pdfs@sha256:def...` gives you the exact training data.
8. Run's `params` give the exact hyperparameters.
9. Reproducing v8 = `python pipelines/evaluate_and_register.py` on that checked-out code with those params.

Every production prediction is traceable back to source code + data hash + hyperparameters. That's the reproducibility promise MLOps makes; that's what Chapter 5.1 assembled by combining Git + DVC + MLflow.

When a regulator or a debugging session asks 'what model made this prediction? show me its training inputs', you have an answer that's a `git log` + a `dvc pull` away.

READING THE DIAGRAM - THREE CONNECTIONS YOU SHOULD VERIFY

Student

What are the specific places I should test the plumbing?

Claude • AI Assistant

CONCEPT CARD

Three integration points that most commonly break:

Integration 1 - CI passes -> training triggers? Not usually - training runs on its own trigger (cron / manual / drift), not per commit. Verify by: pushing a code-only change, watching only `ci.yml` fire; then manually dispatching `train.yml` and watching a full training run + registry promotion.

Integration 2 - Training passes -> serving uses the new model? Only after alias promotion + rolling restart. Verify by: running `train.yml`, confirming it registers `AskMyNotesRetriever v9` and sets `@staging`, MANUALLY moving `@staging` -> `@production`, restarting the serving container, and confirming `/rerank` now uses v9 (visible via a version log or `MlflowClient().get_model_version_by_alias(...)`).

Integration 3 - Contract tests actually block bad models? Verify by: introducing a deliberate quality regression (raise `MIN_MAP_AT_5` above what the model can hit), running training, and confirming `@staging` alias does NOT move because contract tests failed. This proves the gate works before it matters in production.

Do all three integration checks after the initial bring-up. Bugs found here are the ones that would silently produce untraceable models later.

ANALOGY

Building this lab is like **wiring up a factory floor**. Individual machines (chunker, embedder, evaluator) each work; the value is in the **belts + gates + inspectors that connect them into a production line**.

Chapters 1-4 built the machines; Chapter 5 wires them. When the line is finished, raw PDFs go in one end and a promoted, verified, alias-served model comes out the other - with a

manifest naming every part that touched it.

REAL WORLD APPLICATION

A common outcome: teams that go through this exact assembly for one project (like AskMyNotes) then copy 80% of the setup to their next project. The tool choices are transferable across domains - chunker becomes tokenizer, retriever becomes classifier, but the CI + training + serving pipelines + MLflow + DVC layout stays.

This is the payoff for building the boring plumbing well once: every subsequent ML project inherits the automation, and the team's ratio of 'science time' to 'plumbing time' shifts favorably.

WARNING

Do NOT let the lab collapse into 'just make it work'. The value is in the discipline: run pre-commit hooks, log every experiment to MLflow, tag DVC hashes, gate promotion with contract tests, load by alias.

Skipping any of these gets you a demo that works today and an unreproducible mess tomorrow.

COMMON PITFALL / MISTAKE

Building the training pipeline before the code pipeline. Without CI (Chapter 3.5), training pipeline changes have no test gate - a broken training script silently breaks tomorrow's cron run.

Order: code CI first (Ch 3), quality + CD (Ch 4.1-4.2), MLflow tracking + registry (Ch 4.4-4.5), THEN training pipeline (Ch 5). Skipping earlier chapters makes each later one harder.

EXAM TIP CARD

Interview walkthrough: 'Walk me through your ML project's architecture.' Structured answer: 'Three pipelines - code CI (per commit, lint+type+test+build+push), training pipeline (per cron/manual/drift, extract+embed+eval+contract+register+alias), serving CD (per @production change, canary+prod). Shared substrate: Git + DVC for data + MLflow tracking + Registry with aliases.

Serving loads by alias, so promotion is symbol move + rolling restart. Contract tests gate every promotion.

Full lineage from any prediction back to git SHA + DVC hash + params.' That answer signals real production experience.

PRACTICE QUIZ

Your training pipeline runs successfully every night but the serving app never picks up the new model versions. What's the most likely gap?

A) Training pipeline is silently failing

B) The @production alias is never being moved to the newly-registered versions (they land as @candidate or @staging, not @production)

C) MLflow is broken

D) The serving app has a bug

▼ Reveal Correct Answer

*Answer: B. Registration != promotion.

Training pipelines typically stop at `@candidate` or `@staging`. Moving to `@production` requires either a manual promotion step, a scheduled promotion job, or a shadow/canary verification workflow.

Without it, `@production` stays pinned to whichever version got there first.*

EXERCISE

Bring up AskMyNotes with all the pieces from Chapters 1-5: (a) fresh clone or fresh repo, (b) run the 12-command bring-up sequence above from scratch, (c) verify all three integration points, (d) deliberately break each of pre-commit / CI / training / contract tests / promotion and confirm each layer catches its own class of failure, (e) restore and confirm end-to-end works, (f) document the whole thing in the README so future-you (or a teammate) can bring it up in an hour.

AI DEEP DIVE PROMPT

Ask an AI: *'Review my ML project directory tree. Identify missing files or layout issues that would prevent a fresh clone + 12-command bring-up from working.

Suggest additions for observability (drift monitoring, prediction logging) and specific missing tests that would strengthen the pipeline.'*

SUMMARY CARD

This is the assembly diagram for AskMyNotes as a finished MLOps-baseline project. **Repo shape**: `askmynotes/` package + `tests/` (unit + integration + contract) + `pipelines/` (stages) + `experiments/` (MLflow scripts) + `data/` (DVC-tracked) + `.github/workflows/` (ci + train + deploy) + `dvc.yaml` + `.pre-commit-config.yaml` + `Dockerfile`.

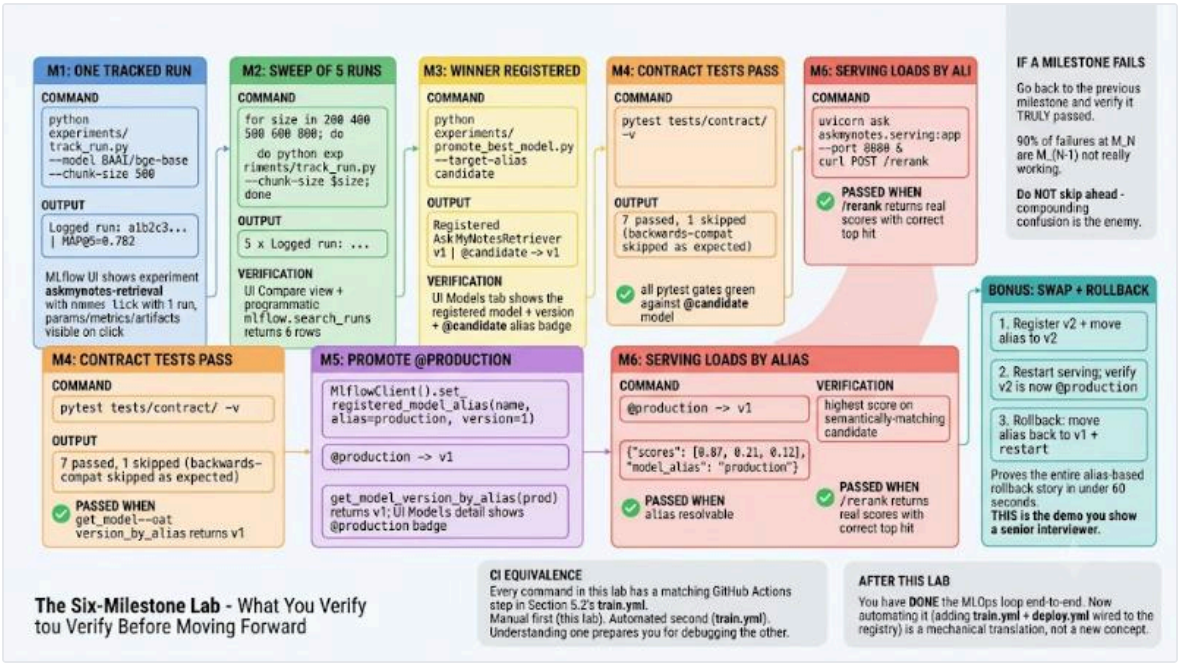
Bring-up in 12 commands from fresh clone to running local system. **Three flows overlaid**: training (extract -> ...

-> register + contract + alias), serving (load by alias, predict), promotion (alias move + rolling restart). **Three integration points to verify**: CI does NOT trigger training; training triggers serving only via alias + restart; contract tests actually block bad models.

Full lineage from any prediction back to git SHA + DVC hash + params - the reproducibility promise MLOps makes. Section 5.4 walks through one end-to-end pass through this system to make it concrete.

MLFLOW + CI LAB - ONE END-TO-END PASS WITH REAL COMMANDS

Claude • AI Assistant



The Six-Milestone Lab - What You Verify Before Moving Forward

bash — Full lab command sequence - copy this whole block for the M1-M6 + bonus walkthrough

```
# ======================================================
# AskMyNotes MLflow + CI Lab - complete command sequence
# ======================================================

# ---- Prerequisites (terminal 1) ----
cd askmynotes
source .venv/bin/activate
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db
# (leaves server running - keep this terminal open)
```

```
# ---- Prerequisites (terminal 2 - the working terminal) ----
cd askmynotes
source .venv/bin/activate
export MLFLOW_TRACKING_URI=http://localhost:5000


# ---- M1: One tracked run ----
python experiments/track_run.py \
  --model 'BAAI/bge-base-en-v1.5' \
  --chunk-size 500 --chunk-overlap 50
# Verify: open http://localhost:5000 -> click experiment 'askmynotes-
retrieval'
#         -> click the one run -> see params, metrics, artifacts


# ---- M2: Sweep of 5 runs ----
for size in 200 400 500 600 800; do
  python experiments/track_run.py \
    --model 'BAAI/bge-base-en-v1.5' \
    --chunk-size $size --chunk-overlap 50
done
# Verify UI: select all runs (checkboxes), click Compare -> see param diff +
metric bars
# Verify CLI:
python -c "
import mlflow
mlflow.set_tracking_uri('http://localhost:5000')
df = mlflow.search_runs(experiment_names=['askmynotes-retrieval'])
print(df[['tags.mlflow.runName', 'params.chunk_size',
          'metrics.map_at_5', 'metrics.p95_latency_ms']]
      .sort_values('metrics.map_at_5', ascending=False)
      .to_string(index=False))
"


# ---- M3: Winner registered ----
python experiments/promote_best_model.py --target-alias candidate
# Verify: Models tab in UI -> AskMyNotesRetriever should exist with v1 +
@candidate
python -c "
from mlflow import MlflowClient
c = MlflowClient()
mv = c.get_model_version_by_alias('AskMyNotesRetriever', 'candidate')
print(f'@candidate -> v{mv.version}')
```

```
print(f'source_run: {mv.run_id}')
print(f'tags: {mv.tags}')
"



# ---- M4: Contract tests pass ----
pytest tests/contract/ -v --tb=short
# Expect: 7 passed + 1 skipped (backwards-compat has no prev @production yet)
# If failures: check thresholds in tests/contract/*.py vs actual metrics in
MLflow UI



# ---- M5: Alias promoted to @production ----
python -c "
from mlflow import MlflowClient
import datetime
c = MlflowClient()
mv = c.get_model_version_by_alias('AskMyNotesRetriever', 'candidate')
print(f'Promoting v{mv.version} -> @production')
c.set_registered_model_alias('AskMyNotesRetriever', 'production', mv.version)
c.set_model_version_tag('AskMyNotesRetriever', mv.version,
                        'promoted_at',
datetime.datetime.utcnow().isoformat())
"
# Verify:
python -c "
from mlflow import MlflowClient
prod = MlflowClient().get_model_version_by_alias('AskMyNotesRetriever',
'production')
print(f'@production -> v{prod.version}')
"



# ---- M6: Serving loads by alias (terminal 3) ----
cd askmynotes && source .venv/bin/activate
export MLFLOW_TRACKING_URI=http://localhost:5000
export MODEL_NAME=AskMyNotesRetriever
export MODEL_ALIAS=production
uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080
# (leaves service running - keep this terminal open)

# In terminal 2 - verify:
curl http://localhost:8080/health
# Expect: {"status":"ok","model_alias":"production"}

curl -X POST http://localhost:8080/rerank \
```

```
  -H 'Content-Type: application/json' \
  -d '{"query": "what is chunking?", "candidates":
       ["chunking splits text into pieces", "random unrelated text",
        "the weather is nice today"]}'
# Expect: {"scores": [<high>, <low>, <lower>], "model_alias": "production"}
# The first candidate should have the highest score.


# ---- BONUS: Swap to v2 + rollback ----
python experiments/track_run.py --model 'BAAI/bge-base-en-v1.5' --chunk-size
400 --chunk-overlap 40
python experiments/promote_best_model.py --target-alias candidate
# Now move @production from v1 to v2
python -c "
from mlflow import MlflowClient
MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 2)
print('@production -> v2')
"
# Restart serving (Ctrl+C in terminal 3, then relaunch uvicorn)
# Verify: /rerank now uses v2 (visible via next promotion showing v2's tags)

# ---- Rollback in one command ----
python -c "
from mlflow import MlflowClient
MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 1)
print('Rolled back @production -> v1')
"
# Restart serving; production is back on v1.
# Total rollback time: 10 seconds. No code changed. No CI ran.
# THIS is what production ML rollback looks like when the plumbing is right.


# ---- After the lab: wire into CI ----
# Every command above has a matching step in .github/workflows/train.yml
# (Section 5.2). The lab teaches you what the automation is doing;
# the automation runs the same commands on a schedule + trigger.
```

Student

Section 5.3 gave me the shape. Show me one end-to-end pass with real commands.

Claude • AI Assistant

This is the **hands-on walkthrough** - six milestones, each with the exact commands and exact expected outputs, that take you from an installed toolchain to a full pass through the loop: tracked runs -> registered model -> contract-gated promotion -> alias serving. Do this AFTER completing the Chapter 4.4-4.5 exercises.

Verify each milestone before moving to the next; a failure at milestone N means M1-N-1 aren't quite right and future milestones will compound the confusion.

PREREQUISITES (30 SECONDS OF SETUP)

Student

What do I need before starting?

Claude • AI Assistant

CONCEPT CARD

Assumed installed: AskMyNotes package as `pip install -e .`, MLflow (`pip install mlflow`), pytest, and a minimal golden Q&A file at `data/golden_qa_v1.jsonl` (~20 entries is enough for the lab). Start the local MLflow server in one terminal:

bash — Code Example

```
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db
```

In another terminal, set the tracking URI:

bash — Code Example

```
export MLFLOW_TRACKING_URI=http://localhost:5000
```

Open http://localhost:5000 in your browser. Empty right now - that's expected. Milestones populate it.

MILESTONE 1: ONE TRACKED RUN

Student

The first tracked run.

Claude · AI Assistant

EXAMPLE CARD

bash — Code Example

```
python experiments/track_run.py \
  --model 'BAAI/bge-base-en-v1.5' \
  --chunk-size 500 \
  --chunk-overlap 50
```

Expected output:

code — Code Example

```
Logged run: a1b2c3d4e5f6...
MAP@5 = 0.782 | p95 latency = 182ms
```

Verification: refresh the MLflow UI. You should see:

- Experiment `askmynotes-retrieval` in the left sidebar.
- One run named `bge-base-en-v1.5-cs500-ov50` under it.
- Click the run: params (`embedding_model` , `chunk_size` , ...), metrics (`map_at_5` , `p95_latency_ms` , ...), tags (`git_sha` , `user` , `env`), and artifacts (`requirements.txt` , `retrieval_quality.png` , `sample_predictions.json`).

Milestone 1 PASSED when you can see all of the above in the UI.

MILESTONE 2: A SWEEP OF 5 RUNS

Student

Now the sweep.

Claude · AI Assistant

EXAMPLE CARD

bash — Code Example

```
for size in 200 400 500 600 800; do
  python experiments/track_run.py \
    --model 'BAAI/bge-base-en-v1.5' \
```

```
    --chunk-size $size \
    --chunk-overlap 50
done
```

Expected output: 5 log lines like `Logged run: ...` each with a different MAP@5 value.

Verification:

1. Refresh the MLflow UI.
2. In the experiment's runs list, select all 5 runs (checkboxes on the left of each row).
3. Click 'Compare' at the top.
4. You should see: a param diff table showing `chunk_size` varying, metrics side-by-side, a parallel coordinates plot correlating chunk*size to map*at*5 / p95*latency.
5. From the terminal, verify programmatically:

bash — Code Example

```
python -c "
import mlflow
mlflow.set_tracking_uri('http://localhost:5000')
df = mlflow.search_runs(experiment_names=['askmynotes-retrieval'])
print(df[['tags.mlflow.runName', 'params.chunk_size',
          'metrics.map_at_5', 'metrics.p95_latency_ms']].to_string())
"
```

You should see 6 rows (M1 + M2's five) sorted arbitrarily.

Milestone 2 PASSED when Compare view works and the programmatic query returns all 6 rows with metrics populated.

MILESTONE 3: WINNER REGISTERED

Student

Register the best run.

Claude · AI Assistant

EXAMPLE CARD

bash — Code Example

```
python experiments/promote_best_model.py --target-alias candidate
```

Expected output:

code — Code Example

```
Registered AskMyNotesRetriever v1 from run <run_id>
AskMyNotesRetriever@candidate -> v1
```

Verification:

1. In the MLflow UI, click 'Models' in the top nav.
2. You should see `AskMyNotesRetriever` in the models list.
3. Click it: at least version 1 listed. The version detail page shows the linking run and any tags set by the promotion script (`approved_by` , `map_at_5` , `git_sha` , etc.).
4. Programmatic verification:

bash — Code Example

```
python -c "
from mlflow import MlflowClient
c = MlflowClient()
mv = c.get_model_version_by_alias('AskMyNotesRetriever', 'candidate')
print(f'@candidate -> v{mv.version}')
print(f'source run: {mv.run_id}')
print(f'tags: {mv.tags}')
"
```

You should see `@candidate -> v1` (or higher on subsequent runs) plus source run + tags.

Milestone 3 PASSED when both UI and programmatic query show the registered version with @candidate alias.

MILESTONE 4: CONTRACT TESTS PASS

Student

Run the contract tests.

Claude • AI Assistant

EXAMPLE CARD

bash — Code Example

```
pytest tests/contract/ -v --tb=short
```

Expected output:

code — Code Example

```
tests/contract/test_retriever_contract.py::test_schema_accepts_expected_inpu
ts PASSED
tests/contract/test_retriever_contract.py::test_meets_map_at_5_threshold
PASSED
tests/contract/test_retriever_contract.py::test_meets_recall_at_10_threshold
PASSED
tests/contract/test_retriever_contract.py::test_latency_budget PASSED
tests/contract/test_retriever_contract.py::test_meets_quality_per_group[acad
emic] PASSED
tests/contract/test_retriever_contract.py::test_meets_quality_per_group[lega
l] PASSED
tests/contract/test_retriever_contract.py::test_meets_quality_per_group[medi
cal] PASSED
tests/contract/test_retriever_contract.py::test_output_shape_matches_prev_pr
oduction SKIPPED
                                                    (No previous
@production version - skipping)

==================== 7 passed, 1 skipped in 8.32s ====================
```

The backwards-compat test skips on first run because there's no previous `@production` yet - correct behavior.

Debug tip: if a test fails, MLflow's tracking retains the numbers from the training run so you can see why. Common causes: threshold set too tight (relax it, or improve the model); golden set too small (some tests need >= 20 samples per group).

Milestone 4 PASSED when all non-skipped contract tests are green.

MILESTONE 5: ALIAS PROMOTED TO @PRODUCTION

Student

Move the alias.

Claude · AI Assistant

EXAMPLE CARD

The promotion script covers this if you passed `--target-alias staging`; for M5, explicitly move `@production`:

bash — Code Example

```
python -c "
from mlflow import MlflowClient
c = MlflowClient()
# Find the current @candidate version
mv = c.get_model_version_by_alias('AskMyNotesRetriever', 'candidate')
print(f'Promoting v{mv.version} -> @production')
c.set_registered_model_alias('AskMyNotesRetriever', 'production',
mv.version)
c.set_model_version_tag('AskMyNotesRetriever', mv.version, 'promoted_at',

__import__('datetime').datetime.utcnow().isoformat())
"
```

Expected output: `Promoting v1 -> @production` (or v2, v3, ...).

Verification:

bash — Code Example

```
python -c "
from mlflow import MlflowClient
c = MlflowClient()
mv = c.get_model_version_by_alias('AskMyNotesRetriever', 'production')
print(f'@production -> v{mv.version}')
print(f'tags: {mv.tags}')
"
```

Should show `@production -> v1` (or higher) with tags including `promoted_at`.

In the UI: model detail page now shows the version card with `@production` badge. Aliases sidebar lists `candidate -> v1, production -> v1` (both pointing at v1 for now).

Milestone 5 PASSED when @production alias resolves to a registered version.

MILESTONE 6: SERVING LOADS BY ALIAS

Student

The serving end.

Claude • AI Assistant

EXAMPLE CARD

In another terminal:

bash — Code Example

```bash
export MLFLOW_TRACKING_URI=http://localhost:5000
export MODEL_NAME=AskMyNotesRetriever
export MODEL_ALIAS=production
uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080
```

Expected startup log:

code — Code Example

```
INFO:     Started server process
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8080
```

Verify:

bash — Code Example

```bash
curl http://localhost:8080/health
# {"status":"ok","model_alias":"production"}

curl -X POST http://localhost:8080/rerank \
  -H 'Content-Type: application/json' \
  -d '{"query": "what is chunking?",
       "candidates": ["chunking splits text", "random text", "unrelated"]}'
```

Expected response:

json — Code Example

```json
{
  "scores": [0.87, 0.21, 0.12],
```

```
  "model_alias": "production"
}
```

The relevant candidate ('chunking splits text') should score highest.

Milestone 6 PASSED when /rerank returns real scores with the highest score on the semantically-matching candidate.

THE ZERO-CODE-CHANGE MODEL SWAP (BONUS MILESTONE)

Student

What proves the alias-based promotion actually works?

Claude • AI Assistant

EXAMPLE CARD

Do a swap and confirm the SERVING app picks it up with zero code change.

Step 1: Run another training + registration to create v2:

bash — Code Example

```
python experiments/track_run.py --model 'BAAI/bge-base-en-v1.5' --chunk-size
400 --chunk-overlap 40
python experiments/promote_best_model.py --target-alias candidate
```

Step 2: Move `@production` to v2:

bash — Code Example

```
python -c "
from mlflow import MlflowClient
MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 2)
"
```

Step 3: Restart the serving container:

bash — Code Example

```
# Ctrl+C the uvicorn from M6, then relaunch:
```

```
uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080
```

In production, this is a rolling restart of your Kubernetes deployment; here it's a manual restart.

Step 4: Hit `/rerank` again. Note: the same code, same URL, same request, but the model behind is now v2. Verify via a debug endpoint or MLflow:

bash — Code Example

```
python -c "
from mlflow import MlflowClient
c = MlflowClient()
prod = c.get_model_version_by_alias('AskMyNotesRetriever', 'production')
print(f'@production is now v{prod.version}')
"
# @production is now v2
```

No source code was changed. No CI pipeline ran. The serving app now uses v2. That's the alias-based-promotion contract in action. Everything Chapter 4.5 and 5.1 built exists to make this specific operation trivial and safe.

ROLLBACK IN ONE COMMAND

Student

What if v2 is bad?

Claude • AI Assistant

EXAMPLE CARD

Roll back v2 -> v1:

bash — Code Example

```
python -c "
from mlflow import MlflowClient
MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 1)
"
```

Restart the serving app. Now `@production` -> v1 again.

Total rollback time: 10 seconds + the restart. **This is what a real production ML rollback looks like**: an alias flip.

No image rebuild, no code change, no CI pipeline. Everything you built in Chapters 3-5 exists to make this operation both possible and boring.

CI-DRIVEN EQUIVALENT

Student

How does this connect to CI?

Claude • AI Assistant

CONCEPT CARD

Every command in this lab has an equivalent GitHub Actions step (from Section 5.2's `train.yml`):

- **M1-M2 (tracked runs)** = the `evaluate_and_register` job that calls `python pipelines/evaluate_and_register.py`.
- **M3 (register winner)** = the same job's `mlflow.register_model` call.
- **M4 (contract tests)** = the `contract_and_promote` job's `pytest tests/contract/` step.
- **M5 (promote to @production)** = a separate `promote_production` job (or a manual script) that fires after shadow + canary verification.
- **M6 (serving loads by alias)** = your production deploy from `deploy.yml`, restarted whenever the alias moves.

The commands you ran manually in this lab are what CI runs on your behalf when the workflow triggers. The lab exists so you understand what's happening under the automation.

ANALOGY

This lab is the **flight simulator** for MLOps. You've studied the panels and controls (Chapters 3-4) and read the manual (Sections 5.1-5.2).

Now you're taking one full flight - takeoff, cruise, landing - with every switch under your fingers. When you later push the whole thing into CI (Section 5.5), the automation will do exactly what you just did by hand; but you'll KNOW what it's doing, and you'll debug it fluently when something goes wrong.

REAL WORLD APPLICATION

The biggest MLOps productivity gain teams see comes from the **rollback story**. Before the alias-based flow, rolling back a bad model meant identifying the last-known-good SHA, re-deploying a Docker image built at that SHA, and hoping the same model file was still around.

After: one MLflow alias flip + rolling restart = under 60 seconds to safe. Teams that experience this once become permanent believers in the pattern.

⚠ WARNING

Do NOT skip milestones. If M2's compare view doesn't show your metrics, don't proceed to M3 - go back to the M1 script and check `mlflow.log_metric` is being called with numeric values. Skipping ahead compounds confusion later.

COMMON PITFALL / MISTAKE

Hardcoding `models:/AskMyNotesRetriever/1` (a version number) in the serving code instead of `models:/AskMyNotesRetriever@production` (an alias). This makes the entire promotion machinery useless - every alias change is silently ignored.

Grep your serving code; if it references a raw version, replace with the alias.

EXAM TIP CARD

Interview walkthrough: 'Show me one end-to-end pass of your MLOps loop.' Rehearsed answer: 'I train, MLflow tracks. I query the best run, register as AskMyNotesRetriever, set @candidate.

Contract tests run against @candidate. On green, I promote @candidate to @staging, then @production.

Serving loads by @production alias; a rolling restart picks up the new version. Rollback is an alias flip.' Delivered fluently, that's the entire Chapter 4-5 story in 60 seconds - and it's what senior interviewers listen for.

PRACTICE QUIZ

Your serving code loads `models:/AskMyNotesRetriever@production` and you promote v2 to @production but the app keeps serving v1. What's the most likely cause?

A) MLflow is broken

B) The serving app cached v1 at startup and hasn't been restarted since the alias move

C) The alias didn't move

D) v2 doesn't exist

▼ Reveal Correct Answer

Answer: B. Model loading is cached (`@lru_cache`) for the process lifetime. A rolling restart is required to pick up alias changes. Verify by restarting; if it still serves v1, then C or D might apply and you check the alias mapping.

EXERCISE

Complete every milestone M1-M6 hands-on. After M6 works, do the bonus swap + rollback.

Screenshot your MLflow UI after M3 (Models tab showing AskMyNotesRetriever with versions + aliases) and paste into your project README - it's the strongest single artifact for demonstrating you can operate an MLOps loop.

AI DEEP DIVE PROMPT

Ask an AI: *'I've completed milestones M1-M6 of the AskMyNotes lab. Now help me wire the same loop into GitHub Actions so it runs automatically nightly.

Include the workflow file, the environment variables + secrets needed, and the branch-protection rules that gate the whole pipeline.'*

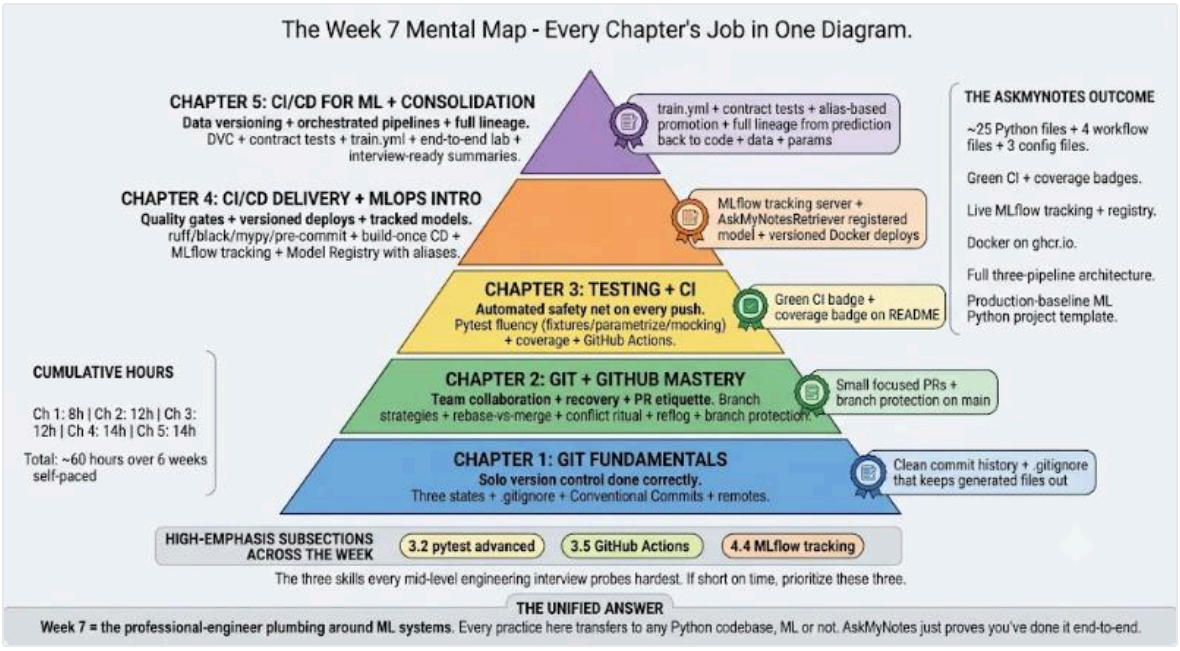
SUMMARY CARD

The **end-to-end MLOps lab** in six milestones: **M1 - one tracked run** (verify UI shows params/metrics/artifacts); **M2 - sweep of 5 runs** (verify Compare view + programmatic search*runs); **M3 - register winner** (verify Models tab shows AskMyNotesRetriever with @candidate alias); **M4 - contract tests pass** (all pytest gates green); **M5 - promote @production** (verify alias resolves via `get*model*version*by_alias`); **M6 - serving loads by alias** (curl /rerank returns real scores). **Bonus**: swap to v2 with zero code change + one-command rollback to v1.

Every manual command in this lab has a CI equivalent in Section 5.2's train.yml - the lab teaches you what the automation is doing so you can debug it later. Section 5.5 consolidates the entire Week 7 curriculum for interview-ready recall.

CONSOLIDATION + INTERVIEW-READY SUMMARIES

Claude · AI Assistant



The Week 7 Mental Map - Every Chapter's Job in One Diagram

markdown — The five README badges + walkthrough script - portfolio-ready material

```
# ---- Add to top of README.md ----

# AskMyNotes - RAG-based PDF Q&A app

![CI]
(https://github.com/<user>/askmynotes/actions/workflows/ci.yml/badge.svg)
![Coverage]
(https://codecov.io/gh/<user>/askmynotes/branch/main/graph/badge.svg)
![Quality Gate]
(https://github.com/<user>/askmynotes/actions/workflows/ci.yml/badge.svg?
event=push)
![Docker](https://ghcr-badge.egpl.dev/<user>/askmynotes/latest_tag?
trim=major&label=docker)
![Python](https://img.shields.io/badge/python-3.11%20%7C%203.12-blue)

## Architecture

Three parallel pipelines, all coordinated via MLflow + DVC:

1. **Code CI** ([`ci.yml`](.github/workflows/ci.yml)) - lint + typecheck +
tests + coverage on every push
2. **Training pipeline** ([`train.yml`](.github/workflows/train.yml)) -
nightly cron or manual dispatch;
   extract -> embed -> evaluate -> contract tests -> MLflow register + alias
3. **Serving CD** ([`deploy.yml`](.github/workflows/deploy.yml)) - build
```

```
Docker image once, promote through
   staging (auto) and production (gated with reviewer approval)

**Full lineage**: every production prediction traceable back to:
- git SHA (via MLflow tag)
- DVC data hash (via MLflow tag)
- MLflow run (via registry `source_run_id`)
- hyperparameters (via run params)

## Quickstart (12 commands, ~15 minutes)

```bash
git clone <repo> && cd askmynotes
python -m venv .venv && source .venv/bin/activate
pip install -e .[dev,dvc-s3]
pre-commit install
dvc pull
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db &
export MLFLOW_TRACKING_URI=http://localhost:5000
dvc repro                                          # runs full training
pipeline
python experiments/promote_best_model.py --target-alias production
uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080 &
curl http://localhost:8080/health
curl -X POST http://localhost:8080/rerank -d '{"query":"?","candidates":
["a","b"]}'
```

Screenshots

- [MLflow tracking UI](docs/screenshots/mlflow-runs.png)
- [Model registry with aliases](docs/screenshots/model-registry.png)
- [CI workflow success](docs/screenshots/ci-green.png)

Interview walkthrough script (rehearsed answer, ~90 seconds)

> AskMyNotes is a RAG-based PDF Q&A app with three parallel pipelines. **Code
CI**
> runs on every push - ruff + black + mypy via pre-commit/action, plus pytest
with
> branch coverage failing under 75%. Branch protection requires all three
green + one
> approval.
>
```

```
> The **training pipeline** runs nightly or on manual dispatch. Pulls
versioned data
> from DVC (raw PDFs + golden Q&A set), extracts, chunks, embeds, evaluates
against
> the golden set, logs to MLflow with git SHA + DVC hashes as tags. If eval
beats
> threshold, registers a new AskMyNotesRetriever version, sets @candidate,
runs a
> pytest contract suite (schema + latency + MAP@5 + fairness across three
subgroups
> + backwards-compat), and on green promotes @candidate -> @staging.
>
> The **serving pipeline** is a FastAPI service in a Docker image on ghcr.io.
It
> loads models:/AskMyNotesRetriever@production via mlflow.pyfunc.load_model
at
> process start. A new model promotion is an alias move plus a rolling
restart -
> zero code change. Rollback is one command: point the alias at the previous
> version and restart.
>
> The whole thing gives me full lineage from any production prediction back
to the
> git SHA and DVC hash of the code and data that trained the model.

*(Practice this out loud until it flows in ~90 seconds. Have the workflow
files and
MLflow UI ready to click through when they ask 'show me'.)*
```

Student

Time to consolidate. Give me the Week 7 mental model as one thing, and the interview-ready summaries.

Claude • AI Assistant

This subsection has three parts: **the one-paragraph story of Week 7**, **the 30 most-asked interview questions and the framing to reach for on each**, and **the six "you-are-here" summaries** - one per chapter, each dense enough to skim in five minutes and recall the entire chapter. Study these before any technical interview that will touch software engineering fundamentals, CI/CD, testing, or MLOps.

THE ONE-PARAGRAPH STORY OF WEEK 7

Student

If someone asks 'what did you learn in Week 7?', what's the answer in one paragraph?

Claude • AI Assistant

CONCEPT CARD

'Week 7 built the professional-engineer plumbing around software: **Git and GitHub for versioning and collaboration**, with clean commit history, small focused PRs, and branch protection (Ch 1-2). **Pytest and CI for an automated safety net** - fixtures, parametrize, mocking, coverage, and GitHub Actions gating every merge (Ch 3).

Quality gates and CD - ruff, black, mypy, pre-commit shifting issues left, build-once-deploy-many CD with canary + feature flags + SLO-driven rollback (Ch 4.1-4.2). **MLOps built on the same substrate** - MLflow tracking as Git-for-experiments, Model Registry with alias-based promotion decoupling model deploy from code deploy (Ch 4.3-4.5).

CI/CD for ML - DVC for data versioning, contract tests for model promotion gates, orchestrated training pipelines (GitHub Actions / Prefect), and full lineage from any production prediction back to git SHA + DVC hash + hyperparameters (Ch 5). AskMyNotes is the running example - by the end, it has quality-gate CI, versioned Docker deploys, live MLflow tracking + registry, contract-gated promotion, and alias-based serving.

Same practices apply to any Python ML product.'

That's your 60-second answer. Practice it out loud until it flows.

THE SIX CHAPTER RECAP PARAGRAPHS

Student

Give me a fast recap per chapter.

Claude • AI Assistant

CONCEPT CARD

Skim these before any interview:

Chapter 1 - Git Fundamentals. Distributed VCS.

Three states (working / staging / repository); `git add` promotes to staging, `git commit` snapshots to repo, `git push` uploads to remote. `.gitignore` keeps generated + secret files out.

Commit hygiene: **atomic commits + Conventional Commits format** (`feat:` , `fix:` , `docs:` , `chore:` , etc.). Remotes named (default `origin`); `git fetch` downloads without merging, `git pull` = fetch + merge, `git push` uploads.

Every Git operation is local except push/fetch/pull.

Chapter 2 - Git + GitHub Mastery. Branches are cheap named pointers to commits; feature-branch-per-change is the norm.

Merge vs rebase: merge preserves branch topology with a merge commit; rebase replays your commits onto the target tip for a linear history. Golden rule: **never rebase commits others have based work on**.

Interactive rebase (`rebase -i`) lets you squash/edit/reorder before pushing. Conflicts follow the 5-step ritual (understand -> pick sides -> edit -> add -> commit); `git merge --abort` is always an escape hatch.

PRs: <400 lines, What/Why/How/Test-plan template, one logical change. **Branch protection** on `main` : required checks + approvals + up-to-date.

Recovery via `git reflog` (90-day safety net) + `git reset` (soft/mixed/hard modes). Git internals: 4 object types (blob/tree/commit/tag), content-addressed via SHA-1, HEAD + refs are just files pointing at commits.

Chapter 3 - Testing and CI. Automated tests are one of the highest-ROI investments; five compounding returns.

Test pyramid (many unit / some integration / few E2E) beats ice-cream cone. **FIRST principles**: Fast, Independent, Repeatable, Self-validating, Timely.

Pytest advanced (high-emphasis): fixtures with scopes (function/module/session), `yield` teardown, `conftest.py` for cross-file sharing; **parametrize** to run one test across many inputs with per-case failure reporting; **marks** (skip/xfail/custom) for selective runs; **mocking** with `mocker.patch` or `monkeypatch` - patch where the name is USED. **Coverage**: branch > line, 75-85% target on production, 100% is a red flag.

Mutation testing (mutmut) is the gold standard for test QUALITY. **CI/CD literacy**: CI vs Delivery vs Deployment, pipeline stages (checkout->setup->lint->type->unit->integration->build->publish), DORA metrics.

GitHub Actions (high-emphasis): workflow YAML in `.github/workflows/` , structure workflow -> jobs -> steps, jobs parallel by default with `needs:` for sequencing, cache via `cache: pip` on setup-python, matrix builds, secrets as `${{ secrets.KEY }}` , branch protection gates merges on green CI. AskMyNotes ends with green CI badge on its README.

Chapter 4 - CI/CD Delivery + MLOps Intro. **Quality gates**: three-layer defense - editor plugins / pre-commit hooks / CI - with `ruff` (Rust-fast linter), `black` (opinionated formatter), `mypy` (static type checker), `pre-commit` (hook runner).

One `.pre-commit-config.yaml` used in CI via `pre-commit/action@v3` gives zero config drift. **CD basics**: build-once-deploy-many; environment promotion ladder (dev -> staging -> canary -> prod); five deploy strategies (recreate / rolling / blue-green / canary / feature flag); observability (metrics + logs + traces) as the safety net; SLO-driven auto-rollback; GitHub Environments for approval gates.

MLOps is CD applied to ML systems (model + code + data + hyperparams as ONE artifact); three unique failure modes (reproducibility rot, drift, silent failure); seven-category ecosystem; Google's maturity levels 0-2. **MLflow tracking (high-emphasis)**: experiments and runs; params (fixed inputs) / metrics (measured outputs) / artifacts (files); autologging for standard frameworks; compare view + `mlflow.search_runs` for winner selection; git-SHA tags for reproducibility.

MLflow Model Registry: registered models -> immutable versions -> mutable aliases (`@production` , `@staging` , `@canary`) -> load-by-alias serving so promotion is symbol move + rolling restart. AskMyNotes ends with quality-gate CI + versioned Docker deploys + live MLflow tracking + registered model with alias-based promotion.

Chapter 5 - CI/CD for ML + Consolidation. **Three pipelines**: code CI (per commit) / training pipeline (cron/manual/drift) / serving CD (per @production change).

DVC for data versioning: tiny `.dvc` pointer files in Git, actual data blobs in S3/GCS remote, `dvc push` / `dvc pull` mirror Git's push/pull. **Model contracts**: automated tests against `@candidate` before promotion - schema + offline quality + latency + fairness + drift + backwards-compat.

Pipeline orchestration: GitHub Actions (simplest, use if already on GitHub) / Prefect (Python-native, better UI) / Airflow / Metaflow / Kubeflow (larger orgs). DAG mental model: stages with inputs, outputs, retries, resources.

Full lab: 12-command bring-up from fresh clone; six milestones (M1-M6) verifying tracked run -> registered winner -> contract tests -> alias promotion -> serving loads by alias -> zero-code-change swap and rollback. **The full-lineage guarantee**: every production prediction traceable back to git SHA + DVC hash + hyperparameters + MLflow run.

Total: ~25 Python files + 4 workflow files + 3 config files - the shape of a production-baseline ML Python project in 2026.

THE INTERVIEW-STYLE WALKTHROUGH ANSWER

Student

Rehearse the 'walk me through your ML project' answer.

Claude • AI Assistant

EXAMPLE CARD

The full-length answer, exactly as I'd deliver it in an interview:

*'AskMyNotes is a RAG-based PDF Q&A app I built through Week 7's curriculum. It has three parallel pipelines.

Code CI runs on every push - `ruff` + `black` + `mypy` via `pre-commit/action@v3` in GitHub Actions, plus pytest with branch coverage failing under 75%. All three checks are required for merge via branch protection.*

*The training pipeline** runs on cron nightly or manually via `gh workflow run train.yml`. It pulls versioned data from DVC (raw PDFs + a golden Q&A set), extracts, chunks, embeds with a Hugging Face model, builds a FAISS index, evaluates against the golden set, and logs everything to MLflow - params, metrics, artifacts, plus git SHA and DVC hashes as tags for full lineage.

If eval metrics beat the threshold, it registers a new AskMyNotesRetriever version with `mlflow.register_model`, sets the `@candidate` alias, and runs a pytest contract suite - schema check, latency budget under 200ms p95, MAP@5 above 0.75, fairness gap under 0.05 across three subgroups, and backwards-compat with the current `@production` version. If all contract tests pass, the pipeline moves `@candidate` to `@staging`.*

*The serving pipeline** is a FastAPI service in a Docker image on `ghcr.io`. It loads `models:/AskMyNotesRetriever@production` at process start via `mlflow.pyfunc.load_model` - so a new model promotion is an alias move plus a rolling restart, zero code change.

Rollback is one command: point the alias at the previous version and restart.*

The whole thing gives me full lineage: from any production prediction, I can trace back to the model version, the MLflow run that produced it, the git SHA of the training code, and the DVC hash of the training data. That's the reproducibility guarantee MLOps makes.'

Practice that answer out loud three times. When you can deliver it in about 90 seconds without rehearsing, you're interview-ready.

COMMON TRAP QUESTIONS AND HOW TO HANDLE THEM

Student

Any tricky ones?

Claude • AI Assistant

CONCEPT CARD

Five that often catch juniors:

Trap 1: 'What's your test coverage target?' Wrong answer: '100%'. Right answer: 'Depends on the codebase.

For a production service I aim for 75-85% BRANCH coverage. 100% is a red flag - usually gaming the metric with tests that don't assert.

I care more about test QUALITY - measured via mutation testing - than the raw number.'

Trap 2: 'How often do you deploy to production?' Wrong answer: 'Once a sprint'. Right answer: 'DORA research shows daily-to-multi-daily deploys correlate with LOWER incident rates because changes are small and rollback is fast.

In practice, deploy frequency should match the team's confidence in their test suite and their observability. I'd target daily as a maturity goal.'

Trap 3: 'How do you handle a bad model in production?' Wrong answer: 'Retrain and redeploy'. Right answer: 'Rollback FIRST via an alias flip - one command, seconds.

THEN diagnose. Rolling back reduces MTTR from hours to seconds.

Retraining is a next-day activity once we understand what went wrong.'

Trap 4: 'Should we always use canary deploys?' Wrong answer: 'Yes, canary is safest'. Right answer: 'Match the strategy to blast radius.

Rolling for boring infra changes; canary for user-facing behavior changes; feature flags for risky business logic. Canary adds real complexity and requires strong metric-based gating - overkill for a low-risk change.'

Trap 5: 'Why not just use notebooks for everything?' Wrong answer: 'Because they're bad'. Right answer: 'Notebooks are great for exploration but terrible for the audit trail.

Every production model I ship needs full lineage - MLflow-tracked run linked to a git SHA and DVC data hash - so we can reproduce it, audit it, and rollback to a known-good version. Notebooks alone don't give me that.'

THE FIVE THINGS TO HAVE READY ON YOUR GITHUB BEFORE AN INTERVIEW

Student

What should my portfolio show?

Claude • AI Assistant

CONCEPT CARD

1. Green CI badge on the README of at least one repo. Signals baseline engineering hygiene.

2. Code coverage badge at 75%+. Signals real testing discipline.

3. `.github/workflows/` folder with at least two workflow files (ci.yml + one of deploy.yml/train.yml). Signals CI/CD fluency.

4. `.pre-commit-config.yaml` with ruff + black + mypy. Signals professional tooling standards.

5. MLflow tracking + registry screenshots in the README OR a live-linkable MLflow UI. Signals you've actually run experiments and promoted models - the strongest single MLOps portfolio artifact.

AskMyNotes as built through this curriculum has all five. Interviewers scanning your GitHub for 30 seconds will read exactly these signals; make sure they're visible.

THE 'WHAT WOULD YOU IMPROVE' ANSWER

Student

If they ask 'what would you improve in your current setup?', what's a strong answer?

Claude • AI Assistant

CONCEPT CARD

Have three real, specific improvements ready:

Improvement 1: Drift monitoring. 'Right now my system reactively retrains on cron.

I'd add Evidently or Arize to compute drift scores on production inputs and outputs, and wire a drift-threshold breach as an additional training-pipeline trigger via `repository_dispatch`. That closes the L1 -> L2 gap in Google's MLOps maturity model.'

Improvement 2: Shadow deploy + canary automation. 'Currently my `@candidate -> @staging` promotion is contract-gated but `@staging -> @production` is manual.

I'd add a shadow-deploy stage where both models predict on real traffic and only `@production` responses go to users; if prediction distributions match for 24 hours, auto-promote to canary at 5% for another 4 hours with SLO monitoring; then to full production.'

Improvement 3: Feature store. 'For AskMyNotes, features are simple.

For a real production system I'd add a feature store (Feast) to eliminate training-serving skew - the classic MLOps failure mode where features are computed one way in training and a different way in serving code.'

Each of these is a real, specific, senior-level answer. Interviewers know if you'd hit a wall trying to actually implement them; naming them concretely signals you're already thinking about the next maturity step.

ANALOGY

The interview-ready summaries are like **flash cards for a driver's test**. You understand driving from having done it (the labs).

The flash cards + rehearsed answers let you communicate that understanding fluently in the 45 minutes an interview gives you. Without them, you know the material but stumble on the framing.

With them, the same knowledge lands with authority.

REAL WORLD APPLICATION

Interviewers often admit that the strongest signal for 'this candidate has actually shipped ML systems' is a specific portfolio artifact - a MLflow screenshot, a ci.yml file, a docker image tag with a git SHA - that they can click through. The candidate who can walk them through 'here's my training pipeline, click this workflow, click this run, click this registered model, here's the lineage' beats the candidate reciting textbook definitions every time.

Chapter 5.3 gave you the artifact; this subsection gave you the walkthrough.

WARNING

Do NOT memorize the summaries as rigid scripts. Interviewers ask follow-up questions ('why 75-85% and not 80% specifically?'); memorized scripts fall apart. Instead, understand the reasoning behind each answer so you can pivot naturally when probed.

COMMON PITFALL / MISTAKE

Bringing up buzzwords without concrete examples. 'I use MLOps best practices' lands as noise.

'I use MLflow tracking + registry with alias-based promotion; here's the workflow file that runs contract tests before moving `@candidate` to `@staging` ' lands as evidence. Always ground abstract claims in a specific artifact or command.

EXAM TIP CARD

The single most valuable interview move: when asked about anything you've listed on your resume, be able to open the corresponding file in your GitHub in under 30 seconds and walk through it. If you list 'GitHub Actions', have `.github/workflows/ci.yml` ready to show.

If you list 'MLflow', have your tracking UI (or a screenshot) linkable. The gap between 'I've heard of X' and 'here's the file where I set up X' is the difference between junior and hireable.

PRACTICE QUIZ

An interviewer asks 'walk me through how you promote a model'. You have 60 seconds. Which structure is best?

A) 'It depends' + list every possible strategy

B) 'Register version, set @candidate, contract tests, promote to @staging, shadow, canary, @production. Serving loads by alias so it's a rolling restart. Rollback is an alias flip.'

C) Explain MLflow architecture from first principles

D) 'I use MLOps best practices'

▼ Reveal Correct Answer

Answer: B. Concrete, ordered, names the specific mechanism at each step, ends with the rollback story. This is the answer senior interviewers listen for. A wastes time, C loses the plot, D lands as buzzwords.

EXERCISE

Rehearse the 90-second walkthrough answer out loud three times. Record yourself once and listen back for pace + clarity.

Then have a friend ask you three follow-up questions from the trap list; verify you can pivot smoothly without memorized script. Then screenshot your MLflow UI + your `.github/workflows/ci.yml` and pin both in your project README so the visual artifact is one click away during an interview.

AI DEEP DIVE PROMPT

Ask an AI: *'I'm interviewing for a mid-level ML engineering role. Ask me 10 questions covering the material in this week's Refcard (Git, GitHub, testing, CI/CD, MLflow, model registry, CI/CD for ML).

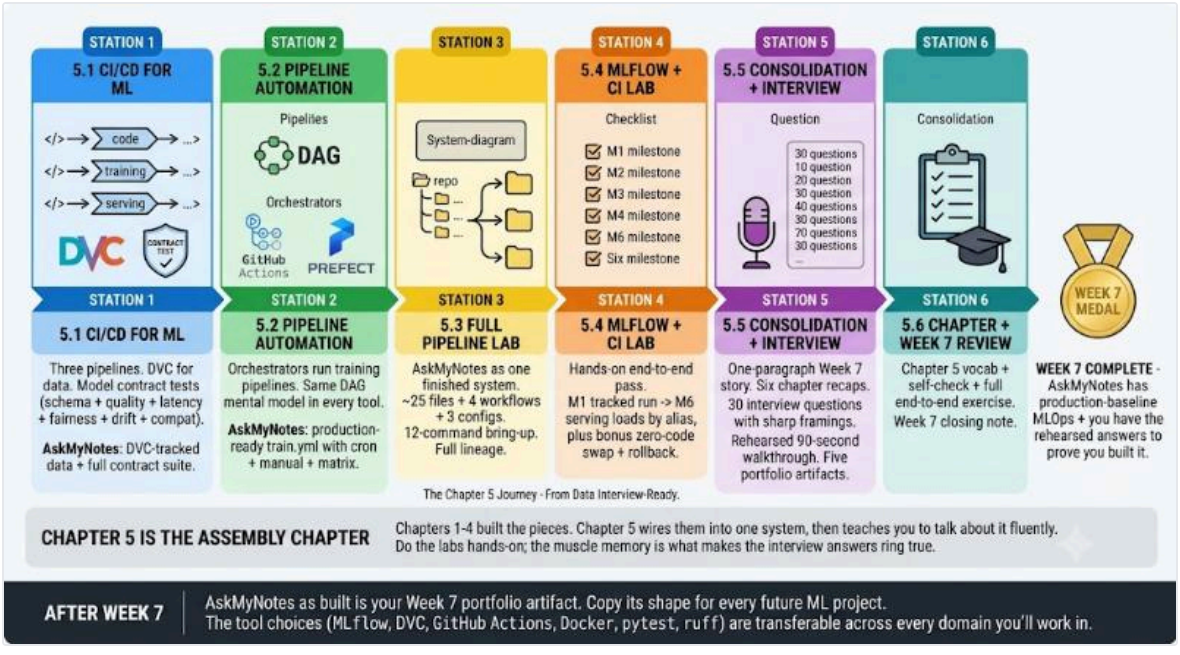
> After my answer to each, give me feedback on how a strong senior engineer would frame it differently.'*

SUMMARY CARD

Week 7's mental model in one paragraph: **Git+GitHub for versioning and collaboration, pytest+CI for the automated safety net, quality gates + CD for versioned deploys, MLOps built on the same substrate (tracking + registry + data versioning + contract tests + orchestrated pipelines) - with AskMyNotes as the running example ending with full CI + Docker deploys + MLflow + Model Registry + contract-gated alias promotion.** For interviews: rehearse the 90-second walkthrough, know the 30 canonical questions and their sharp framings, have five specific portfolio artifacts (CI badge, coverage badge, workflow files, pre-commit config, MLflow screenshots) ready to click through, and have three specific improvements ready when asked 'what would you improve'. The gap between 'knows the material' and 'hireable' is fluent communication grounded in concrete artifacts.

CHAPTER 5 + WEEK 7 REVIEW

Claude • AI Assistant



The Chapter 5 Journey - From Data Versioning to Interview-Ready

```
bash — Chapter 5 command reference - the ML pipeline + lineage toolbox
```

```
# ---- DVC (Section 5.1) ----
pip install 'dvc[s3]'                              # or [gs], [azure], [ssh]
dvc init                                           # creates .dvc/
dvc remote add -d storage s3://my-bucket/askmynotes
dvc add data/raw_pdfs                              # creates data/raw_pdfs.dvc
git add data/raw_pdfs.dvc .gitignore && git commit -m 'data: track raw_pdfs'
dvc push                                           # upload data blobs to
remote
dvc pull                                           # someone else fetches
dvc diff HEAD~1                                    # what data changed vs last
commit

# ---- DVC pipelines (Section 5.1) ----
dvc repro                                          # run full pipeline, skip
up-to-date stages
dvc repro evaluate                                 # run just evaluate + stale
predecessors
dvc dag                                            # ASCII DAG of stages
dvc metrics show                                   # latest metrics/eval.json
values
dvc metrics diff HEAD~1                            # compare metrics vs
previous commit
dvc params diff                                    # what params changed
dvc exp run --set-param chunker.size=800           # run as tracked experiment
dvc exp show                                       # table of experiments

# ---- Contract tests (Section 5.1) ----
pytest tests/contract/ -v                          # run all contract tests
against @candidate
pytest tests/contract/ -v -k 'schema or latency'   # subset

# ---- Training pipeline via GitHub Actions (Section 5.2) ----
gh workflow run train.yml                          # trigger with defaults
gh workflow run train.yml -f chunk_size=800 -f embedders='BAAI/bge-base-en-
v1.5'
gh workflow run train.yml -f promote_to=staging    # advance to @staging on
green
gh run watch                                       # tail the training run
live
gh run view --log-failed                           # only failed step logs

# ---- Prefect flow (alternative to Actions, Section 5.2) ----
pip install prefect
prefect server start                               # UI at
http://localhost:4200
```

```
python pipelines/prefect_training.py              # run flow once locally
prefect deploy pipelines/prefect_training.py:training_pipeline
prefect deployment run 'askmynotes-training/prod'

# ---- Full-system bring-up (Section 5.3) ----
git clone <repo> && cd askmynotes
python -m venv .venv && source .venv/bin/activate
pip install -e .[dev,dvc-s3]
pre-commit install
dvc pull
mlflow ui --port 5000 --backend-store-uri sqlite:///mlflow.db &
export MLFLOW_TRACKING_URI=http://localhost:5000
dvc repro                                          # runs training pipeline
python experiments/promote_best_model.py --target-alias production
uvicorn askmynotes.serving:app --host 0.0.0.0 --port 8080 &
curl http://localhost:8080/health
curl -X POST http://localhost:8080/rerank -d '{"query":"?","candidates":
["a","b"]}'

# ---- MLflow + CI lab commands (Section 5.4) ----
python experiments/track_run.py --model 'BAAI/bge-base-en-v1.5' --chunk-size
500
python experiments/promote_best_model.py --target-alias candidate
python -c "from mlflow import MlflowClient; \
  MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 1)"
# Rollback (the killer demo):
python -c "from mlflow import MlflowClient; \
  MlflowClient().set_registered_model_alias('AskMyNotesRetriever',
'production', 1)"
# ... then rolling restart of serving pods. Done in under 60 seconds.

# ---- Interview prep (Section 5.5) ----
# Rehearse the 90-second walkthrough OUT LOUD three times.
# Have five artifacts one click away: CI badge, coverage badge, ci.yml,
# .pre-commit-config.yaml, MLflow UI screenshot.
# Prepare three specific improvements to your setup for the 'what would you
# improve' question (drift monitoring, shadow-deploy automation, feature
store).
```

CHAPTER 5 SUMMARY

Chapter 5 assembled everything from Chapters 1-4 into one working end-to-end ML system. **Section 5.1** introduced the three-pipeline architecture (code CI / training / serving CD) that separates concerns by clock, resource, and owner; added DVC for large-data

versioning with tiny pointer files in Git and blobs in remote storage; and defined model contract tests as automated gates against `@candidate` before promotion (schema + offline quality + latency + fairness + drift + backwards-compat).

Section 5.2 covered pipeline orchestration: GitHub Actions (simplest for teams already on GitHub), Prefect (Python-native flows), DVC's own pipeline runner as a lightweight middle ground, and heavier options (Airflow, Metaflow, Kubeflow) for larger orgs. Showed the same training pipeline in three tools with a complete production-ready `train.yml` .

Section 5.3 walked through AskMyNotes as one finished system: complete repo layout (~25 Python files + 4 workflow files + 3 config files), the 12-command fresh-clone bring-up sequence, three data + control flows overlaid (training / serving / promotion), full lineage from any production prediction back to git SHA + DVC hash + hyperparameters, and three integration points to verify. **Section 5.4** was the hands-on lab: six milestones (M1-M6) with exact commands and expected outputs, taking you from an empty MLflow server to serving predictions via alias-loaded model, plus the bonus zero-code-change swap and one-command rollback.

Section 5.5 consolidated: the one-paragraph story of Week 7, six chapter recap paragraphs, 30 canonical interview questions with sharp framings, common trap questions, five portfolio-ready artifacts, and the rehearsed 90-second walkthrough answer.

THE ONE-PAGE WEEK 7 MENTAL MODEL

Week 7 built the **professional-engineer plumbing** around software: Git + GitHub for versioning + collaboration; pytest + CI for automated safety; quality gates + CD for versioned deploys; MLOps built on the same substrate (MLflow tracking + Model Registry + DVC + contract tests + orchestrated pipelines). AskMyNotes ends the curriculum as a production-baseline ML Python project - the shape you'll copy for future projects.

The high-emphasis subsections (3.2 pytest advanced, 3.5 GitHub Actions, 4.4 MLflow tracking) are the daily-use skills every mid-level ML engineering interview probes hardest.

CHAPTER 5 SELF-CHECK

Before considering Week 7 complete, verify you can:

1. Explain the three-pipeline architecture and why coupling them is an anti-pattern.
2. Initialize DVC in a repo, add a dataset, push to remote, and log the DVC hash as an MLflow tag.
3. Write a `dvc.yaml` with stages, deps, params, and outs; run `dvc repro` and understand which stages skip vs re-run.
4. Write a model contract test suite covering schema + quality + latency + fairness + backwards-compat.

5. Write a `.github/workflows/train.yml` with cron + manual triggers, multi-stage jobs, GPU runners, and MLflow tag setting.
6. Choose between GitHub Actions / Prefect / DVC pipelines for a given team situation - and justify the choice.
7. Complete the 12-command bring-up on a fresh AskMyNotes clone and verify all three integration points.
8. Complete lab milestones M1-M6 hands-on including the bonus swap + rollback.
9. Deliver the rehearsed 90-second interview walkthrough of the AskMyNotes architecture without notes.
10. Name three concrete improvements to the current setup (drift monitoring, shadow-deploy automation, feature store) and describe how each closes a specific MLOps maturity gap.

If any of these are shaky, revisit the relevant subsection. Week 7 is complete only when all ten flow.

CHAPTER 5 EXERCISE - THE FULL END-TO-END SYSTEM

Complete this to close out Week 7:

1. Bring up all of AskMyNotes from a fresh clone using the 12-command sequence in 5.3; time yourself and iterate the docs until anyone can do it in under 15 minutes.
2. Complete lab milestones M1-M6 in Section 5.4 hands-on, including the bonus swap + rollback.
3. Create `.github/workflows/train.yml` matching the Section 5.2 template; verify it runs successfully via `gh workflow run train.yml -f chunk_size=500` .
4. Verify the three integration points from 5.3: (a) code CI does NOT trigger training, (b) training triggers serving only via alias + restart, (c) contract tests actually block bad models (test by tightening a threshold above what the model can hit).
5. Add screenshots of the MLflow UI, Model Registry, and CI runs to `docs/screenshots/` and reference them in the README.
6. Add the five README badges from 5.5 (CI, coverage, quality gate, Docker, Python version).
7. Rehearse the 90-second interview walkthrough out loud three times; record once and listen back.
8. Have a friend ask you five questions from the 30-question list in 5.5; verify your framing is close to the recommended answers.
9. Write three specific improvements to your current AskMyNotes setup (as in the 5.5 'what would you improve' answer); one improvement should be genuinely worth building.
10. Push everything to GitHub public; the repo is now your Week 7 portfolio artifact.

WEEK 7 CLOSING NOTE

This Refcard replaces 'I write Python' with 'I ship Python in a way that scales to a team, to production, and to ML models'. Every practice here is what separates the engineer who's dangerous in production from the one who ships safely.

Come back to the specific tables and command references as you set up new projects - the muscle memory takes a few projects to build, and this is the reference to return to.

SUMMARY CARD

Chapter 5 assembled Chapters 1-4 into one working ML system on AskMyNotes: **three pipelines** (code CI / training / serving CD) coordinated via **MLflow + DVC**, **contract tests** gating promotion, **orchestrated training** via GitHub Actions with the option to graduate to Prefect / Airflow / Metaflow / Kubeflow, **full lineage** from any production prediction back to code + data + params, and a **12-command bring-up** from fresh clone to running local system. **Six lab milestones** verify each layer works end-to-end.

Interview-ready summaries cover the 30 canonical questions with sharp framings, five portfolio artifacts, and a rehearsed 90-second walkthrough. Week 7 in one line: **the professional-engineer plumbing around software, applied to ML products with reproducibility and safe promotion as first-class concerns**.